# User Manual

## Factory Edition – original EBCTester board

Battery testing with ZKE EBC testers – stand-alone, recorded on SD, operated from the browser

| | |
|---|---|
| Firmware | 1.99.3 (V2 beta, 5 November 2026) |
| Hardware | EBCTester board, schematic revision 1, PCB marking B1.1 |
| Firmware mode | FactoryMode (GPIO 47 high) |
| Supported testers | ZKE EBC-A20, EBC-A40L, EBC-A50L |
| Document | Edition November 2026, English |
| Project | github.com/Gemelon/EBCTester |

© 2026 Thomas Stoll. Software and documentation licensed under the Apache License 2.0. EBCTester is an independent hobby project and not a product of, or supported by, ZKETECH.

# Contents

**Part I – Operating Manual** . . . 4

**1 Introduction** . . . 5
- 1.1 What EBCTester does . . . 5
- 1.2 Main features . . . 5
- 1.3 Factory and Maker devices . . . 5
- 1.4 Supported testers . . . 6
- 1.5 What you need . . . 6
- 1.6 Safety information . . . 7

**2 Setting up** . . . 8
- 2.1 Connecting the hardware . . . 8
- 2.2 First start and login . . . 9
- 2.3 Connecting to your home Wi-Fi (optional) . . . 10
- 2.4 Selecting the tester model . . . 10
- 2.5 LEDs . . . 11
- 2.6 The web interface at a glance . . . 11

**3 Battery tester section** . . . 13
- 3.1 Measurements . . . 13
- 3.2 Battery . . . 14
- 3.3 Manual control . . . 16
- 3.4 Automatic control (test plans) . . . 17
- 3.5 Internal resistance (DCIR) . . . 19

**4 Administration section** . . . 23
- 4.1 Device information . . . 23
- 4.2 Test run management . . . 24
- 4.3 SD card . . . 25
- 4.4 Updates . . . 26
- 4.5 Settings . . . 27
- 4.6 Debug . . . 31

**5 Typical workflows** . . . 32
- 5.1 Measuring the capacity of a battery . . . 32
- 5.2 Measuring internal resistance at 50 % state of charge . . . 32
- 5.3 Backing up your data . . . 32

**6 Troubleshooting** . . . 33

**Part II – Technical Reference** . . . 35

**7 System architecture** . . . 36

**8 Start-up and device configuration** . . . 37
- 8.1 Start-up sequence . . . 37
- 8.2 Firmware modes . . . 37
- 8.3 GPIO 47 detection . . . 38
- 8.4 Configuration storage . . . 38
- 8.5 Firmware defaults . . . 38

**9 Hardware** . . . 40
- 9.1 Connectors . . . 40
- 9.2 GPIO assignment of the board . . . 41

**10 Firmware installation** . . . 42
- 10.1 Installing a release . . . 42
- 10.2 Tester interface . . . 42

**11 Data storage** . . . 43

11.1 SQLite database on the SD card . . . 43
11.2 MariaDB backup database . . . 43
**12 Interfaces . . . 44**
12.1 HTTP API . . . 44
12.2 Home Assistant MQTT topics . . . 44
**13 Security and limitations . . . 45**
13.1 Security . . . 45
13.2 Known limitations (firmware 1.99.3) . . . 45
13.3 License . . . 45
**Appendix A – Language files . . . 46**
**Appendix B – Glossary . . . 47**

# Part I – Operating Manual

This part explains how to set up and use an EBCTester with the original board: connecting the tester, logging in, entering batteries, running manual tests, test plans and internal-resistance measurements, and reviewing and managing the recorded data. No programming or electronics knowledge is needed – the device is ready to use as delivered.

# 1 Introduction

## 1.1 What EBCTester does

ZKE (ZKETECH) EBC battery testers are popular, affordable instruments for charging and discharging batteries with constant current, constant power or constant voltage and for measuring capacity. Normally they are controlled by the manufacturer's Windows software, which means a PC has to stay connected for the whole test – often for days.

**EBCTester replaces that PC.** A small controller with an ESP32-S3 microcontroller is connected to the tester's serial interface. It controls the tester, runs complete test plans on its own and records every measurement in a database on a microSD card. You operate it with any web browser – from a phone, a tablet or a PC – either through your home Wi-Fi or through the device's own Wi-Fi access point. You can close the browser at any time: the test continues and keeps being recorded.

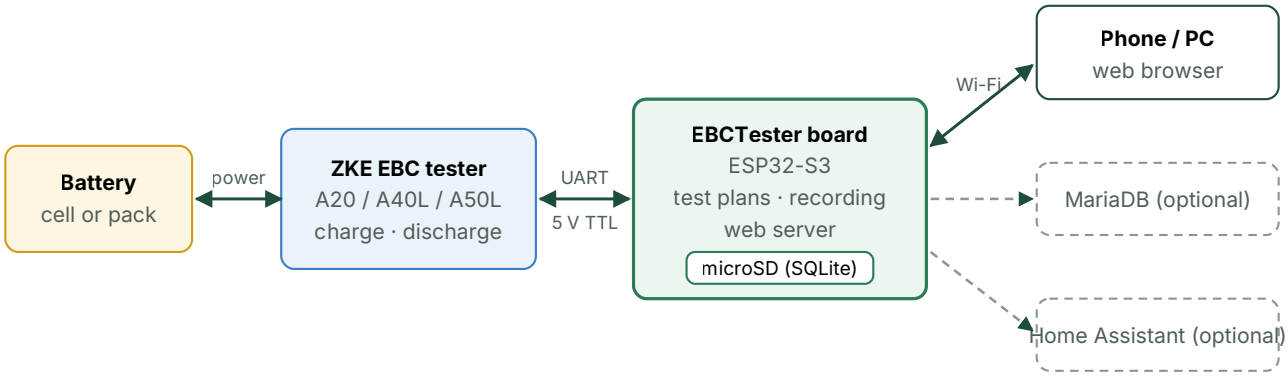



**Figure 1:** System overview

## 1.2 Main features

- **No PC needed during a test.** Manual operations and automatic test plans run on the ESP32-S3.
- **Offline recording.** All measurements go into an SQLite database on a FAT32 microSD card. Recording needs a ready SD card, not an Internet connection.
- **Browser interface.** Charts, battery records, test control and administration. Everything, including the chart library, is served by the device itself; no cloud service and no Internet access are required.
- **Test plans** with up to 10 steps: constant-current discharge, constant-power discharge, CC/CV charge, wait and repeat.
- **Internal resistance (DCIR)** by a load-pulse test with three separate results (by default after 1, 5 and 10 seconds), optionally with automatic preparation to a target state of charge.
- **Test-run management:** search, filter, rename and delete recorded runs.
- **Optional** backup to a MariaDB server (for example on a NAS), Home Assistant integration via MQTT and a live debug log.
- **Manual firmware updates** over Wi-Fi: new versions are announced, but only installed after you confirm.
- **Languages:** English and German built in; additional languages can be loaded from the SD card.

## 1.3 Factory and Maker devices

The same firmware runs on the original EBCTester board and on self-built devices. At the very first start it decides which of two modes applies, based on the level of GPIO 47:

| Mode | Detected when | Behaviour |
|---|---|---|
| FactoryMode | GPIO 47 is pulled high (original board: solder jumper JP1 closed to 3.3 V, as delivered) | Starts immediately with the pin assignment of the original board. All functions are available at once. |
| MakerMode | GPIO 47 is low or not connected | Starts with all hardware outputs switched off. Only the Wi-Fi access point and the web interface run until the maker has entered and activated the hardware configuration in **Maker Settings**. |

This manual describes the **Factory Edition**. If you have built your own device, use the **Maker Edition** of this manual instead – it also covers the configuration of your hardware and contains a detailed technical reference.

The mode is detected only once and then stored. It is detected again only after the device configuration has been reset (see Device reset (section 4.5)).

## 1.4 Supported testers

The firmware recognises a range of ZKE models by the identifier in their status messages. Only the models marked "enabled" can be controlled; all others are shown as read-only telemetry until their operation has been confirmed.

| Model | Max. charge | Max. discharge | Max. power | Status |
|---|---|---|---|---|
| EBC-A20 | 18 V / 5 A | 30 V / 20 A | 85 W | enabled |
| EBC-A40L | 5 V / 40 A | 5 V / 40 A | 200 W | enabled |
| EBC-A50L | 5 V / 50 A | 5 V / 50 A | 250 W | enabled |
| EBC-A05(+), EBC-A10(+/H), EBC-A20+, EBD-M10, EBD-A10H, EBD-M03(+), EBD-M05, EBD-A100(L), EBC-A100L | recognised | | | read-only |

These limits are enforced for every manual command, every plan step and every internal-resistance parameter. The A40L and A50L are low-voltage testers (max. 5 V): use them only with single lithium cells and settings that suit the cell. EBCTester has no battery-chemistry selection – the values you enter must be right for your battery.

> **Note**
> If you own one of the read-only models and can test it with EBCTester, please share your results via the GitHub project so the model can be enabled.

## 1.5 What you need

- An EBCTester board with installed firmware (as delivered).
- A supported ZKE tester and a cable from the board's 6-pin connector J4 to the tester's serial (TTL) port.
- Power: the 19 V supply of the EBC-A20 (through J4) or a USB-C power supply.
- A 2.4 GHz antenna (the ESP32-S3-WROOM-1U module has no on-board antenna).
- A microSD card formatted as FAT32 (exFAT is not supported). 4–32 GB is ideal.
- A CR2032 coin cell for the real-time clock (optional).
- A phone, tablet or computer with a current web browser.

## 1.6 Safety information

> **Warning**
> Batteries store a lot of energy. Wrong settings, damaged cells or poor connections can cause overheating, fire or explosion. Lithium batteries in particular must never be charged above their maximum voltage or discharged below their minimum voltage.

- Enter charge voltages, currents and cut-off values that match the battery's data sheet. EBCTester only checks the limits of the tester, not those of your battery.
- The software does not replace hardware protection (BMS, fuse, temperature monitoring). Long tests are not automatically safe to leave unattended.
- If the serial connection is lost, EBCTester cannot guarantee that the tester stops. Check the tester locally after any connection error.
- Use cables and connectors rated for the test current, and keep the test setup on a non-flammable surface.
- The web interface uses unencrypted HTTP. Use it only in your own trusted network or through the device's protected access point – never expose it to the Internet.

# 2 Setting up

## 2.1 Connecting the hardware

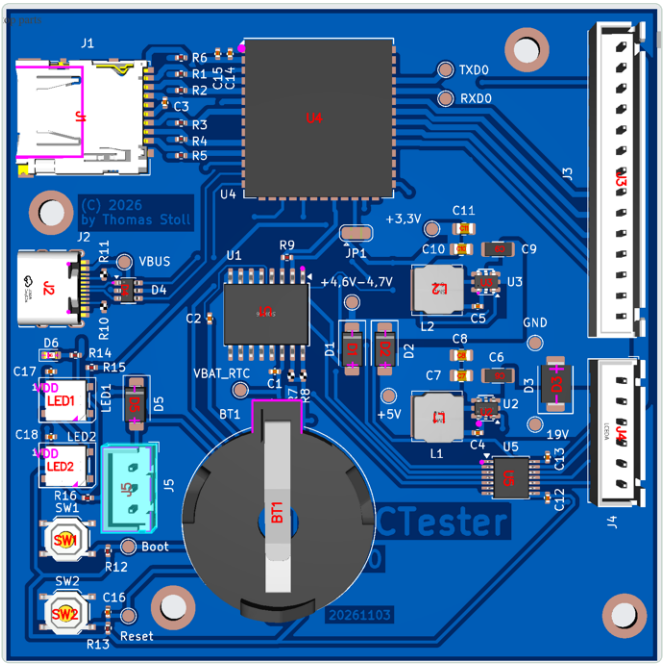



**Figure 2:** EBCTester board (rendering from the production data)

The connectors are on the edges of the board: J4 for the tester at the bottom of the right edge, the expansion connector J3 above it, the microSD socket J1 at the top left and the USB-C socket J2 on the left edge. The LEDs are on the left side, the BOOT and RESET buttons in the bottom left corner and the CR2032 holder in the middle. The antenna socket (u.FL) is on the ESP32 module U4 at the top of the board.

1. **Antenna.** Mount the SMA socket of the antenna cable in the housing, plug its small u.FL plug onto the socket of the ESP32 module and screw on the antenna. Without an antenna Wi-Fi does not work.
2. **microSD card.** Insert a FAT32-formatted card. The card is never formatted automatically; a new, empty database is created on it at first use.
3. **Tester connection.** Connect J4 to the tester's serial interface (pin assignment in Connectors (section 9.1)). The board's TX line goes to the tester's RX line and vice versa. With the EBC-A20, J4 can also supply the board from the tester's 19 V adapter.
4. **Power.** Power the board either through J4 (19 V) or through USB-C. Both may be connected at the same time.

> **Caution**
> The pin order of the cable on the tester side differs between tester models (the EBC-A40L uses a different connector, and TX/RX have been reported swapped compared with the A20 cable). Check the pinout with a multimeter before connecting. The A40L and A50L are mains-powered and do not supply 19 V – power the board through USB-C in that case.

> **Note**
> Leave solder jumper JP1 in its delivered state (pads 1–2 bridged). It tells the firmware that it runs on an original board. Changing it has an effect only after a device reset and turns the device into a Maker device that has to be configured by hand.

## 2.2 First start and login

On first start the device opens its own Wi-Fi access point. Its name is `BatteryTester-XXXXXX`, where `XXXXXX` is unique to the device. EBCTester uses two separate passwords:

| Password | Used for | Initial value |
|---|---|---|
| **Access-point password** | Connecting to the Wi-Fi network `BatteryTester-XXXXXX` | Set when the firmware was built; the standard firmware uses `ebctester`. If the firmware was built without one, a random password is generated at the first start and shown in the start-up log. |
| **Login password** | Logging in to the web interface (user `admin`) and confirming every settings change | `ebctester` |

To read a generated access-point password, connect the device to a PC by USB-C, open a serial terminal (for example the ESP-IDF monitor or PuTTY) and press RESET. The line `Generated setup AP password` shows the password; it stays the same after restarts.

1. Connect your phone or PC to the Wi-Fi network `BatteryTester-XXXXXX` using the access-point password.
2. Open `http://192.168.4.1` in the browser.
3. Log in with user name `admin` and the login password – on a new device `ebctester`. The login page labels this field **Admin password**.
4. Change the login password right away under **Administration → Settings → Login Settings** (see Login Settings (section 4.5)).

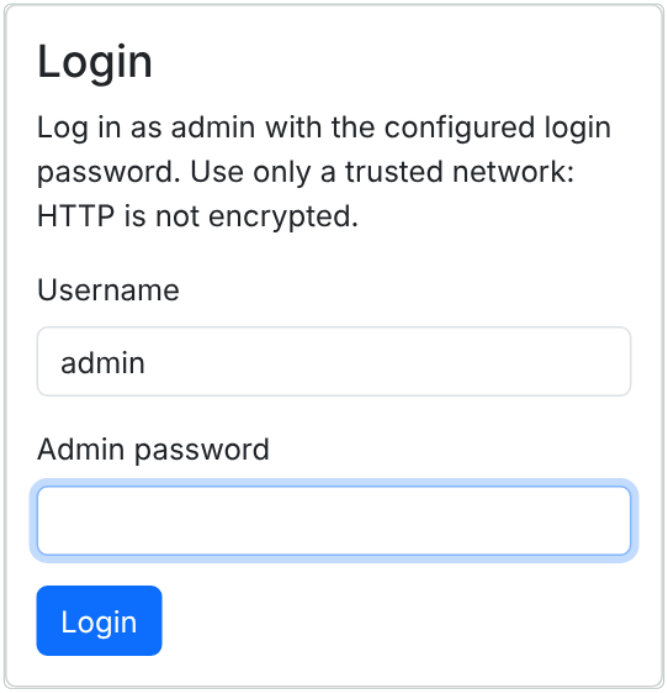



**Figure 3:** Login page

> **Caution**
> `ebctester` is the same on every device and is published in this manual. Change it on first use. A device reset (see Device reset (section 4.5)) restores this initial password.

A login is valid for 24 hours or 7 days, depending on the setting. It is not extended by activity and ends when the device restarts or you click **Log out**. Running tests and recording are not affected by logging in or out. The login can also be switched off completely (see Login Settings (section 4.5)).

## 2.3 Connecting to your home Wi-Fi (optional)

EBCTester works completely without home Wi-Fi. Connecting it to your network is still useful: you can reach it from every device in the house, the clock is synchronised from the Internet (needed for correct date/time stamps and for update checks) and optional services such as MariaDB and MQTT become possible.

1. Go to **Administration → Settings → Configure Wi-Fi**.
2. Enter the network name (SSID) and password and click **Save**. Confirm with the login password when asked.
3. The device connects; its new IP address is shown under **Device information → Wi-Fi IP**. Your router also lists it.

The protected access point remains active as a fallback, so you can always reach the device at `http://192.168.4.1`, even when the home network is out of range. Changing the Wi-Fi channel may briefly interrupt the access-point connection.

## 2.4 Selecting the tester model

Before the first connection you must tell EBCTester which tester you use – there is no default. Go to **Administration → Settings → EBC tester**, select the model and click **Save tester model**.

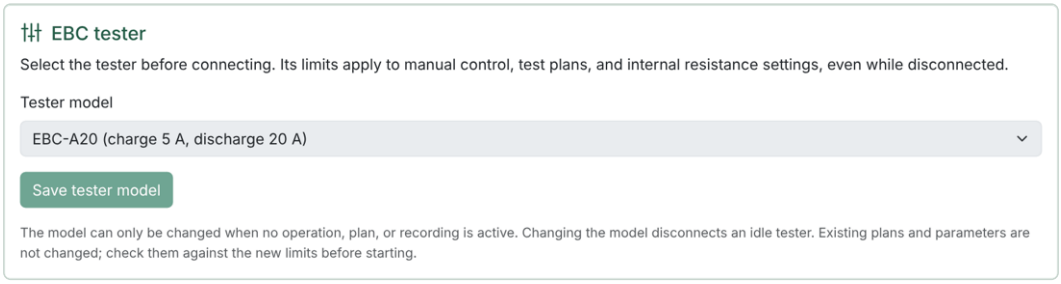



**Figure 4:** Selecting the tester model

The selected model and its limits apply immediately, even without a connected tester, so you can prepare plans and parameters in advance. The model can only be changed while no operation, plan or recording is active. If the connected tester reports a different supported model, EBCTester shows an error, disconnects and does not reconnect automatically until you correct the selection or click **Connect** again.

## 2.5 LEDs

| LED | Display | Meaning |
|---|---|---|
| **D6 – power** | green, steady | The 3.3 V supply is OK. If D6 stays dark, check the 19 V or USB supply. |
| **LED1 – status** | slow colour cycle | Normal operation |
| | blue, pulsing | Firmware update is being downloaded |
| | yellow, pulsing | Downloaded firmware is being verified |
| | green, steady | Update verified, device is about to restart |
| | red, blinking | Firmware update failed |
| **LED2** | off | Not used by firmware 1.99.3 (reserved for a tester activity indicator) |

The header of the web interface shows the communication with the tester (see The web interface at a glance (section 2.6)).

## 2.6 The web interface at a glance

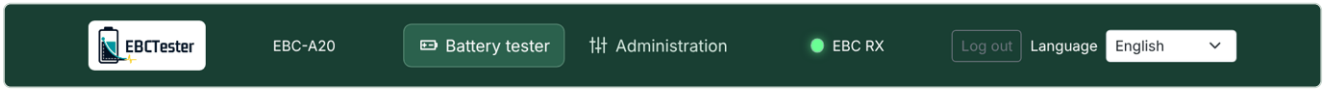



**Figure 5:** Header bar

The header bar is always visible. From left to right it contains:

| | |
|---|---|
| **Logo** | Returns to the start page. |
| **Model** | The selected tester model (for example EBC-A20). It turns into a warning if the connected tester does not match or is unknown. |
| **Battery tester / Administration** | Switches between the two sections of the interface. Each section remembers its last open tab. |

| | |
|---|---|
| **RX indicator** | Green while the tester is connected, with a short flash for every valid message received from the tester. Grey = tester not connected, amber = the browser cannot reach the device. |
| **Log out** | Ends the browser session. Hidden when browser login is switched off. |
| **Language** | English, Deutsch and any languages loaded from the SD card. The choice is stored in the browser; switching reloads the page (unsaved form entries are lost, a running test is not affected). |

| Section | Tabs |
|---|---|
| **Battery tester** | Measurements · Battery · Manual · Automatic · Internal resistance |
| **Administration** | Device information · Test run management · SD card · Updates · Settings · Debug (only when enabled) |

Numeric input fields are checked as soon as you leave them. A red message below the field explains the allowed range; start and save buttons stay disabled until all fields are valid. When a start button is disabled for another reason (for example tester not connected), a yellow note below it tells you why.

> **Note**
> The screenshots in this manual were created with simulated example data: an EBC-A20 testing a 50 Ah LiFePO4 cell. They show example settings, not recommendations for your battery.

# 3 Battery tester section

## 3.1 Measurements

The **Measurements** tab is the dashboard for recorded test runs. All data comes from the database on the SD card, so the page also works on the access point without Internet.

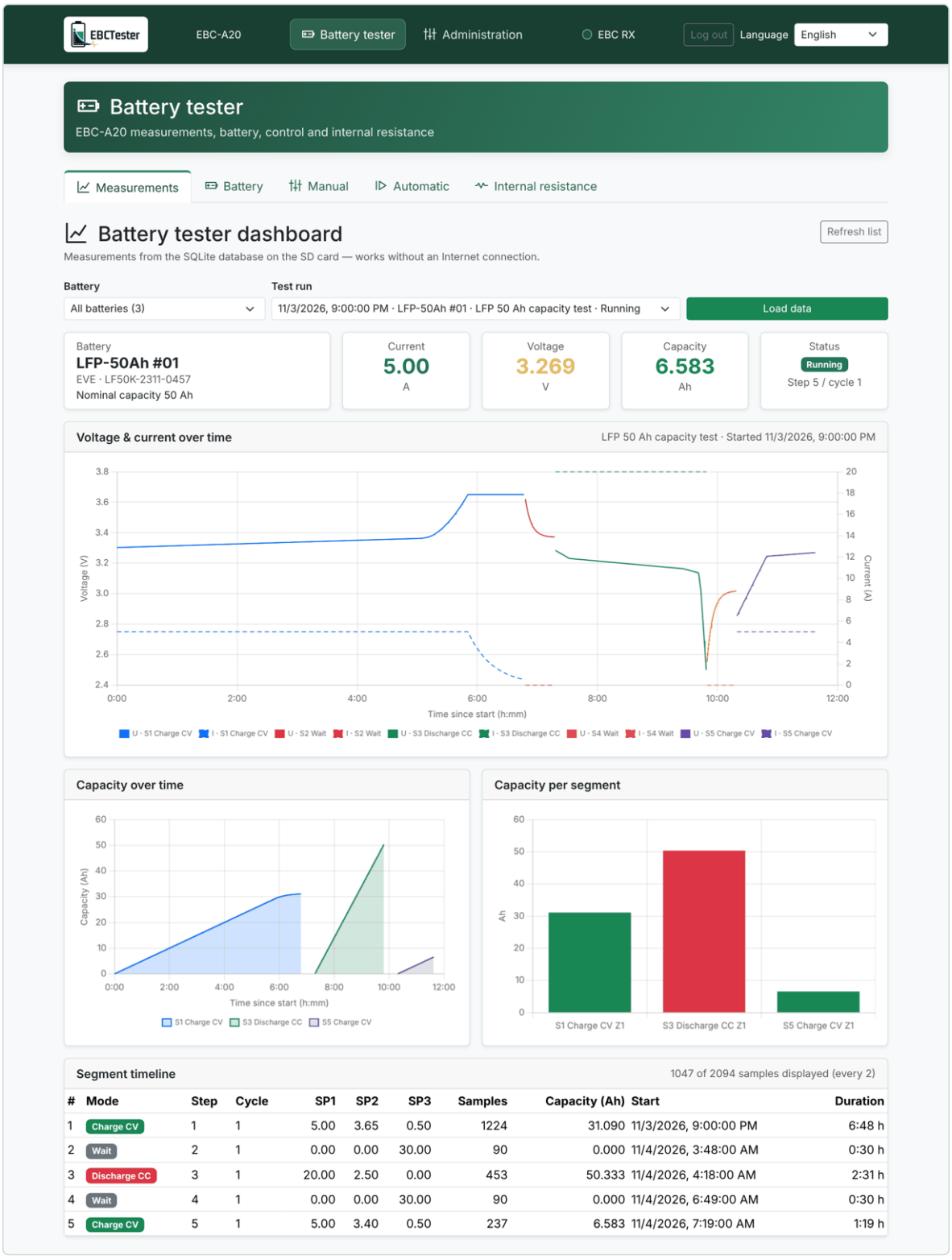



**Figure 6:** Measurements dashboard during a running capacity test

### Selecting a test run

1. Optionally restrict the list with **Battery**.
2. Choose a run under **Test run**. Runs are listed newest first with date, battery, plan and status (or with the custom name given in test-run management).
3. Click **Load data**. **Refresh list** re-reads the list of runs, for example after a new test was started.

The newest run is loaded automatically when the page opens. A running test is refreshed every 15 seconds.

### What is shown

- **Summary cards:** battery (label, manufacturer, serial number, nominal capacity), the latest current, voltage and capacity, and the run status with current step and cycle.
- **Voltage & current over time:** one colour per segment; voltage as solid line (left axis), current as dashed line (right axis). The time axis shows hours:minutes since the start of the run.
- **Capacity over time** for every charge or discharge segment, and **Capacity per segment** as a bar chart (green = charge, red = discharge).
- **Segment timeline:** one row per segment with mode, plan step, cycle, the three set-points (SP1–SP3), number of measurements, capacity, start time and duration.

A *segment* is a continuous section of a run with one operating mode – for example "charge CC/CV" or "wait". To keep the charts fast, at most 1500 points are drawn; the line above the timeline states how many of the stored points are shown ("every n-th"). Segment boundaries are always included. Hover over a chart (or tap it) to see the exact values.

| Status | Meaning |
|---|---|
| **Running** | The test is still being recorded. |
| **Completed** | Finished normally (plan complete or cut-off condition reached). |
| **Stopped** | Stopped by the user. |
| **Interrupted** | Ended by a power loss, restart or lost connection. |
| **Error** | Ended because of an error (see the debug log for details). |

## 3.2 Battery

Here you enter the battery that will be tested next. Its data is stored with every test run, so you always know which cell or pack a measurement belongs to.

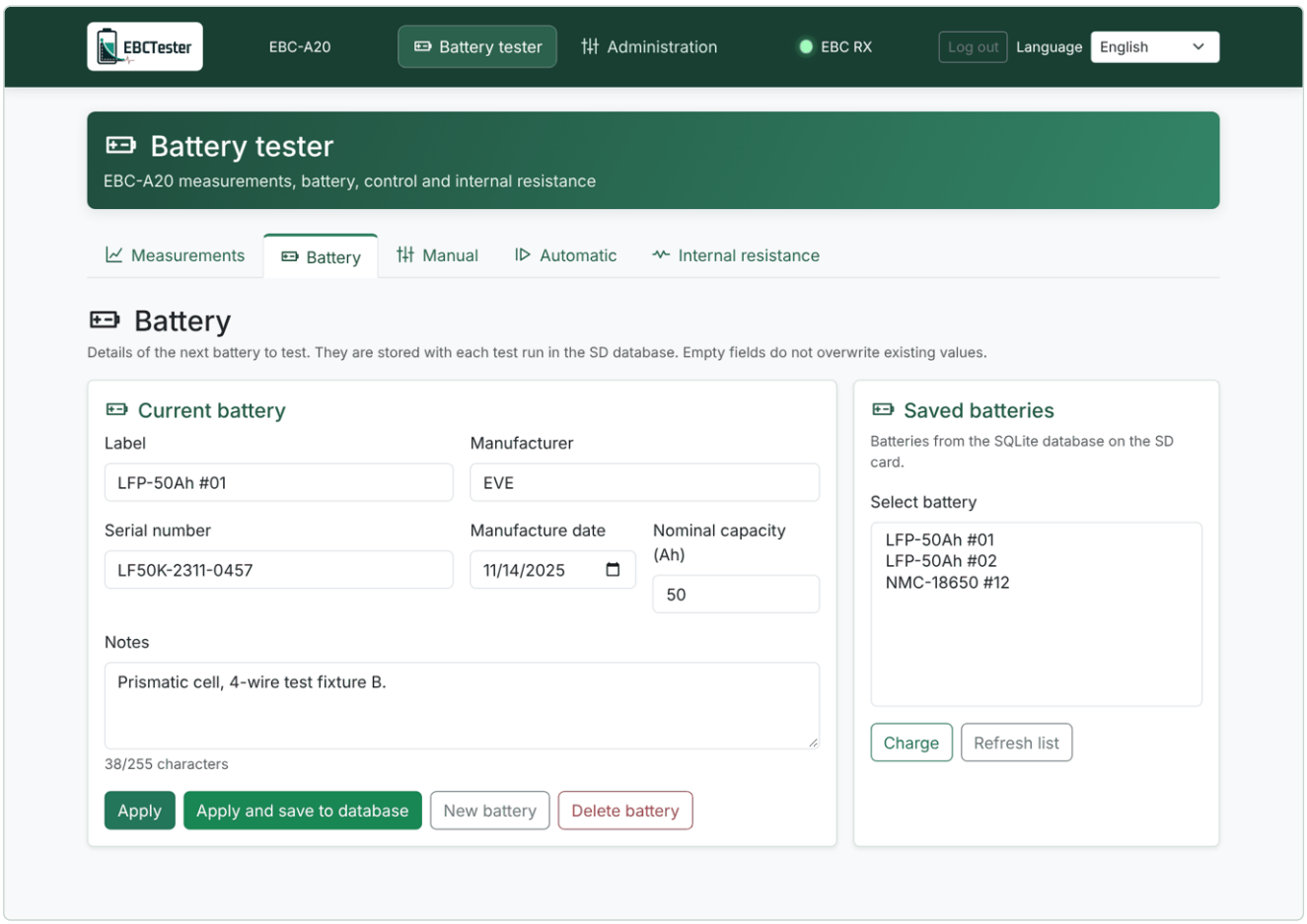



**Figure 7:** Battery tab

| Field | Description |
|---|---|
| **Label** | Unique name of the battery, for example *LFP-50Ah #01*. Required for saving to the database. |
| **Manufacturer, Serial number** | Free text. |
| **Manufacture date** | Date picker. |
| **Nominal capacity (Ah)** | Rated capacity; shown in the dashboard for comparison. |
| **Notes** | Up to 255 characters. |

| Button | Function |
|---|---|
| **Apply** | Makes the entered battery the current battery (stored on the device). The next test run is recorded with it. |
| **Apply and save to database** | Additionally saves the battery in the SD database so it can be selected again later. |
| **New battery** | Clears the form for a new entry. |
| **Delete battery** | Deletes the saved battery shown in the form after confirmation and the login password. Not possible while it is used by recorded runs – delete those runs first in test-run management – or while a recording, plan or DCIR test is active. MariaDB backups are not changed. |

| Button | Function |
|---|---|
| **Load button / Refresh list** | Under **Saved batteries**: select a battery and click the button below the list (labelled **Charge** in firmware 1.99.3) or double-click the entry to load it into the form. **Refresh list** re-reads the database. |

Empty fields do not overwrite values that are already stored.

## 3.3 Manual control

The **Manual** tab gives you direct control of the tester – comparable to the buttons on the tester itself, but with recording.

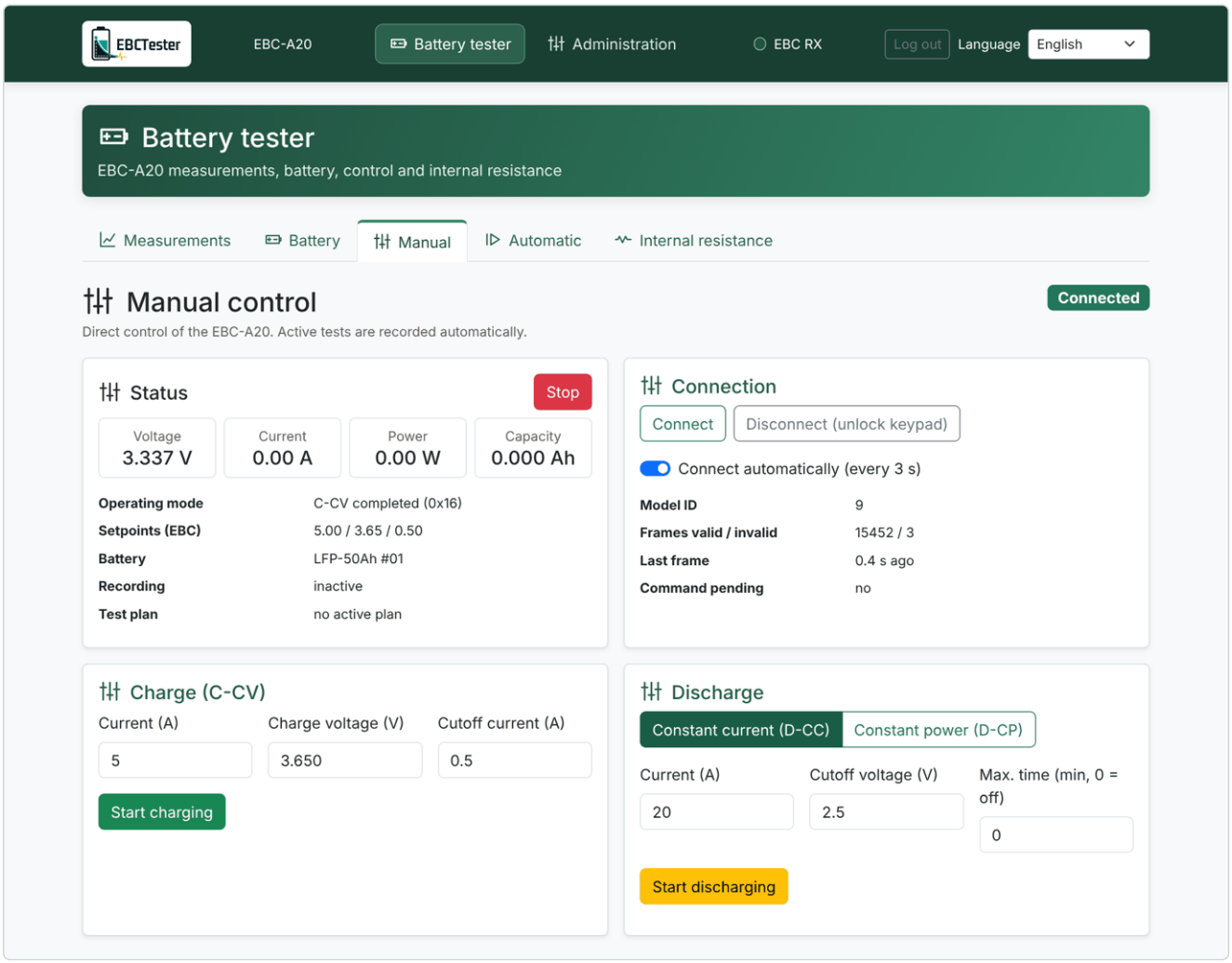



**Figure 8:** Manual control with the tester connected and idle

### Status

Shows the live values from the tester (voltage, current, power, capacity), the operating mode with its code, the set-points currently active in the tester, the current battery, the recording state (number of stored measurements) and whether a test plan is active. The badge at the top right shows *Disconnected*, *Connected* or the active mode. **Stop** stops the current operation immediately; it is always available while connected.

### Connection

- **Connect** starts communication with the tester. The tester's keypad is locked while connected.
- **Disconnect (unlock keypad)** ends the connection and unlocks the keypad.

- **Connect automatically (every 3 s)**: EBCTester tries to connect on its own, for example after switching the tester on.
- Diagnostic values: model ID reported by the tester, number of valid/invalid messages, age of the last message, and whether a command is waiting for confirmation.

### Charge (C-CV)

Charges with constant current until the charge voltage is reached, then with constant voltage until the current falls below the cut-off current.

| Field | Meaning | EBC-A20 range |
|---|---|---|
| **Current (A)** | Charge current in the CC phase | 0.01 – 5 A |
| **Charge voltage (V)** | End-of-charge voltage (CV level) | up to 18 V |
| **Cutoff current (A)** | Charging ends when the current drops below this value | ≤ charge current |

### Discharge

Select **Constant current (D-CC)** or **Constant power (D-CP)**.

| Field | Meaning | EBC-A20 range |
|---|---|---|
| **Current (A) / Power (W)** | Discharge current (D-CC) or power (D-CP) | 0.01 – 20 A / 1 – 85 W |
| **Cutoff voltage (V)** | Discharging ends at this battery voltage | up to 30 V |
| **Max. time (min, 0 = off)** | Additional time limit | 0 – 239 min |

In D-CP mode the power must also be achievable within the maximum current at the cut-off voltage (power ≤ max. current × cut-off voltage). For the A40L/A50L the ranges of the respective model apply.

After **Start charging** or **Start discharging**, EBCTester waits up to 10 seconds for the tester to confirm the mode; otherwise it reports an error and sends Stop. Every manual operation is recorded automatically as a test run.

> **Note**
> Start buttons are disabled while a command is pending, while another operation or a test plan is running (including its wait steps) and while no tester is connected. The reason is shown below the button.

## 3.4 Automatic control (test plans)

A test plan is a sequence of up to 10 steps that EBCTester executes on its own. The plan runs on the ESP32 and continues without Wi-Fi and without an open browser as long as the device is powered.

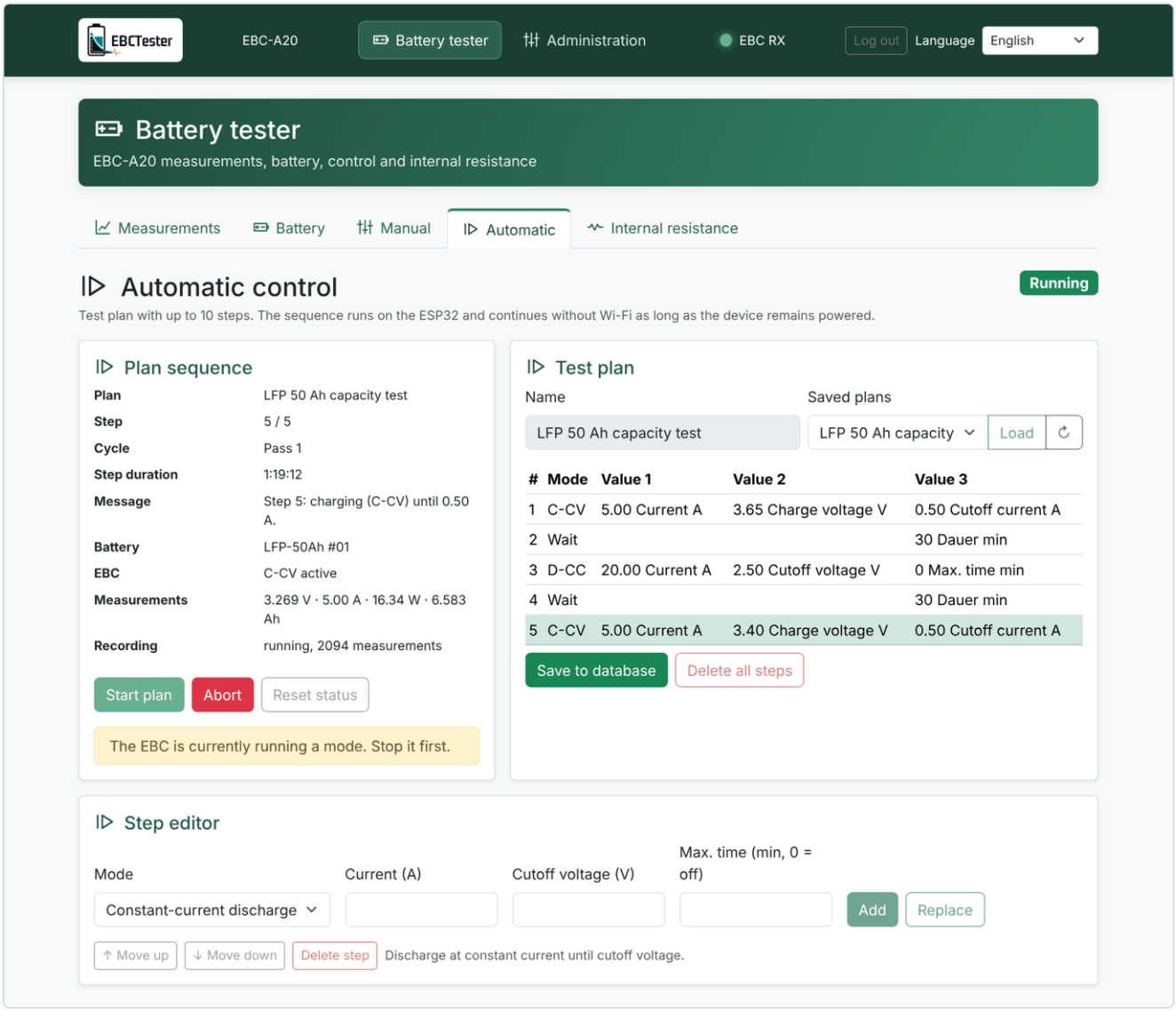



**Figure 9:** Automatic control: running plan, plan table and step editor

## Step types

| Mode | Value 1 | Value 2 | Value 3 | Function |
|---|---|---|---|---|
| **D-CC** | Current (A) | Cutoff voltage (V) | Max. time (min, 0 = off) | Discharge at constant current until the cut-off voltage. |
| **D-CP** | Power (W) | Cutoff voltage (V) | Max. time (min, 0 = off) | Discharge at constant power until the cut-off voltage. |
| **C-CV** | Current (A) | Charge voltage (V) | Cutoff current (A) | Charge CC/CV until the cut-off current. |
| **Wait** | – | – | Duration (1 – 239 min) | Pause, for example to let the cell recover. |
| **Cycle** | – | – | Total passes (1 – 239) | Returns to step 1 until the number of passes is reached. Not allowed as first step and only once per plan. |

## Creating and editing a plan

1. Enter a **Name** for the plan.

2. In the **Step editor**, choose the mode, fill in the values and click **Add**. The step is appended – or inserted after the selected row.
3. Click a row in the table to select it. You can then **Replace** it with the editor values, move it (**↑ Move up**, **↓ Move down**) or **Delete step**. Click the row again to deselect it.
4. **Save to database** stores the plan under its name on the SD card. **Delete all steps** clears the table.

Every change is saved on the device immediately and checked against the limits of the selected tester. Saved plans can be loaded again with the **Saved plans** list and **Load**; ↻ refreshes the list.

### Example: capacity test of a LiFePO4 cell

| # | Mode | Values | Purpose |
|---|---|---|---|
| 1 | C-CV | 5.00 A · 3.65 V · 0.50 A | Charge fully |
| 2 | Wait | 30 min | Let the cell settle |
| 3 | D-CC | 20.00 A · 2.50 V · 0 (no time limit) | Measure the capacity |
| 4 | Wait | 30 min | Let the cell recover |
| 5 | C-CV | 5.00 A · 3.40 V · 0.50 A | Charge to a storage level |

Example values only – use the values from your cell's data sheet.

### Running a plan

Connect the tester, check the current battery and click **Start plan**. The **Plan sequence** card shows the plan name, current step, cycle, time in the step, status message, battery, tester mode, live values and the number of recorded measurements. The active step is highlighted green in the table.

- **Abort** stops the plan and the tester after confirmation.
- **Reset status** returns a completed, aborted or interrupted plan to "Ready" so it can be started again.

> **Caution**
> After a power failure or restart, a running plan is not resumed automatically. It is marked *Interrupted*. Check the battery, then click **Reset status** and start again if required.

## 3.5 Internal resistance (DCIR)

This tab measures the DC internal resistance of a battery with a load pulse. The battery rests without load, then a defined discharge current is switched on. From the voltage drop the resistance is calculated three times – by default 1, 5 and 10 seconds after switching on:

$$R = (U_1 - U_2) / I_2$$

$U_1$ is the average rest voltage at the end of the rest phase, $U_2$ and $I_2$ are the voltage and the measured current at the sampling time. Short times mainly reflect the ohmic resistance (contacts, electrolyte); longer times include polarisation effects.

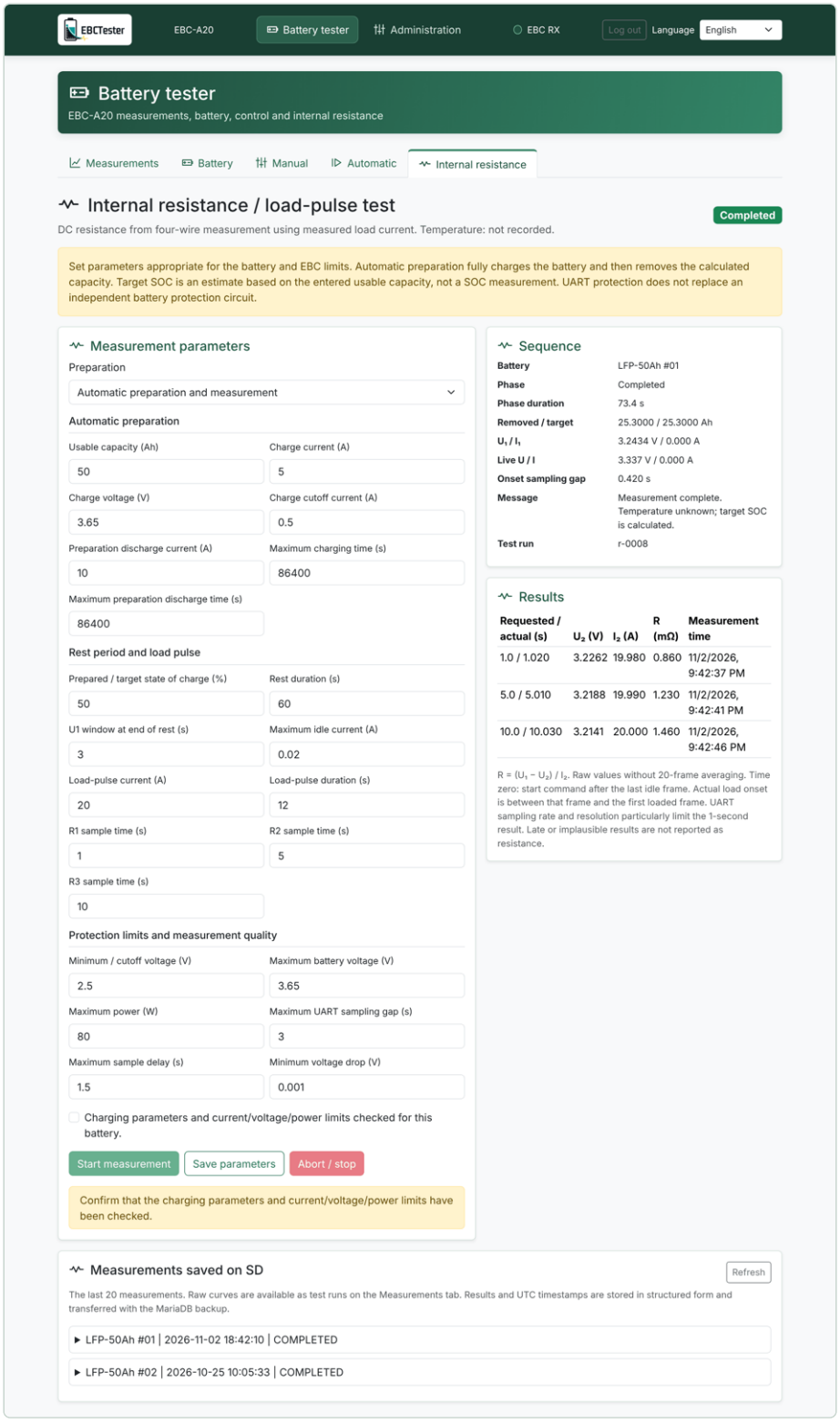



**Figure 10:** Internal resistance: parameters, sequence and results of a completed test

## Preparation methods

- **State of charge already prepared:** the battery is already at the desired state of charge. The test starts with the rest phase.
- **Automatic preparation and measurement:** the battery is first fully charged (C-CV), then the calculated amount of charge is removed: usable capacity × (100 − target SOC) / 100. The removed charge is integrated from the measured current. The target SOC is therefore an estimate based on the entered capacity, not a measured value. A target of 100 % skips the discharge.

### Parameters

| Group / parameter | Meaning | Default |
|---|---|---|
| **Automatic preparation** (only with automatic preparation) | | |
| **Usable capacity (Ah)** | Capacity used to calculate the charge to remove | 20 |
| **Charge current / voltage / cutoff current** | Full charge before the measurement | 0.5 A / 3.45 V / 0.1 A |
| **Preparation discharge current (A)** | Current used to reach the target SOC | 0.5 |
| **Max. charge / preparation time (s)** | Time-outs for both phases | 86400 |
| **Rest and load pulse** | | |
| **Prepared / target SOC (%)** | Target state of charge (automatic) or information (prepared) | 50 |
| **Rest time (s)** | Rest without load before the pulse | 60 |
| **U1 window at end of rest (s)** | Averaging window for $U_1$ | 3 |
| **Max. idle current (A)** | Current that still counts as "no load" | 0.02 |
| **Pulse current (A) / duration (s)** | Load pulse | 0.5 A / 12 s |
| **Sample times R1 / R2 / R3 (s)** | Times of the three results after switching on | 1 / 5 / 10 |
| **Protection limits and measurement quality** | | |
| **Minimum / cutoff voltage (V)** | Test stops below this voltage | 2.5 |
| **Maximum battery voltage (V)** | Test stops above this voltage | 3.6 |
| **Maximum power (W)** | Upper limit for current × max. voltage | 50 |
| **Max. UART gap (s)** | Longest permitted interval between tester messages | 3 |
| **Max. sample delay (s)** | Timing tolerance of the samples | 1.5 |
| **Min. voltage drop (V)** | Smaller drops are rejected as not credible | 0.001 |

Consistency rules: sample times must increase; the pulse must last at least as long as the last sample time plus the timing tolerance; the rest time must be at least the U1 window; cut-off and charge voltage must be below the maximum voltage; and current × maximum voltage must not exceed the maximum power. In addition, all values must be within the limits of the selected tester.

### Running the measurement

1. Connect the battery with four-wire sensing (separate sense leads) – otherwise the cable resistance is included in the result.
2. Check all parameters and click **Save parameters**.
3. Tick the confirmation box ("Charging parameters and current/voltage/power limits checked for this battery"). Changing any parameter clears this box again.
4. Click **Start measurement**. The SD database must be ready.
5. Follow the progress in the **Sequence** card; **Abort / stop** ends the test at any time.

The **Results** table lists for each sampling time the requested and actual time, $U_2$, $I_2$, the resistance in mΩ and the time stamp. Late or missing samples, too small or negative voltage drops or missing current produce an "invalid" result instead of a wrong number. *Onset sampling gap* shows the uncertainty of the switch-on moment caused by the interval between tester messages – this mainly limits the accuracy of the 1-second value.

Every DCIR test is stored as a normal test run (raw curve in Measurements) plus structured results. **Measurements saved on SD** lists the last 20 DCIR tests. Temperature is not measured and is reported as unknown.

> **Note**
> During a DCIR test all other starts, connect/disconnect and battery changes are blocked. Stop and Abort always remain available. A DCIR test is never resumed after a restart.

# 4 Administration section

## 4.1 Device information

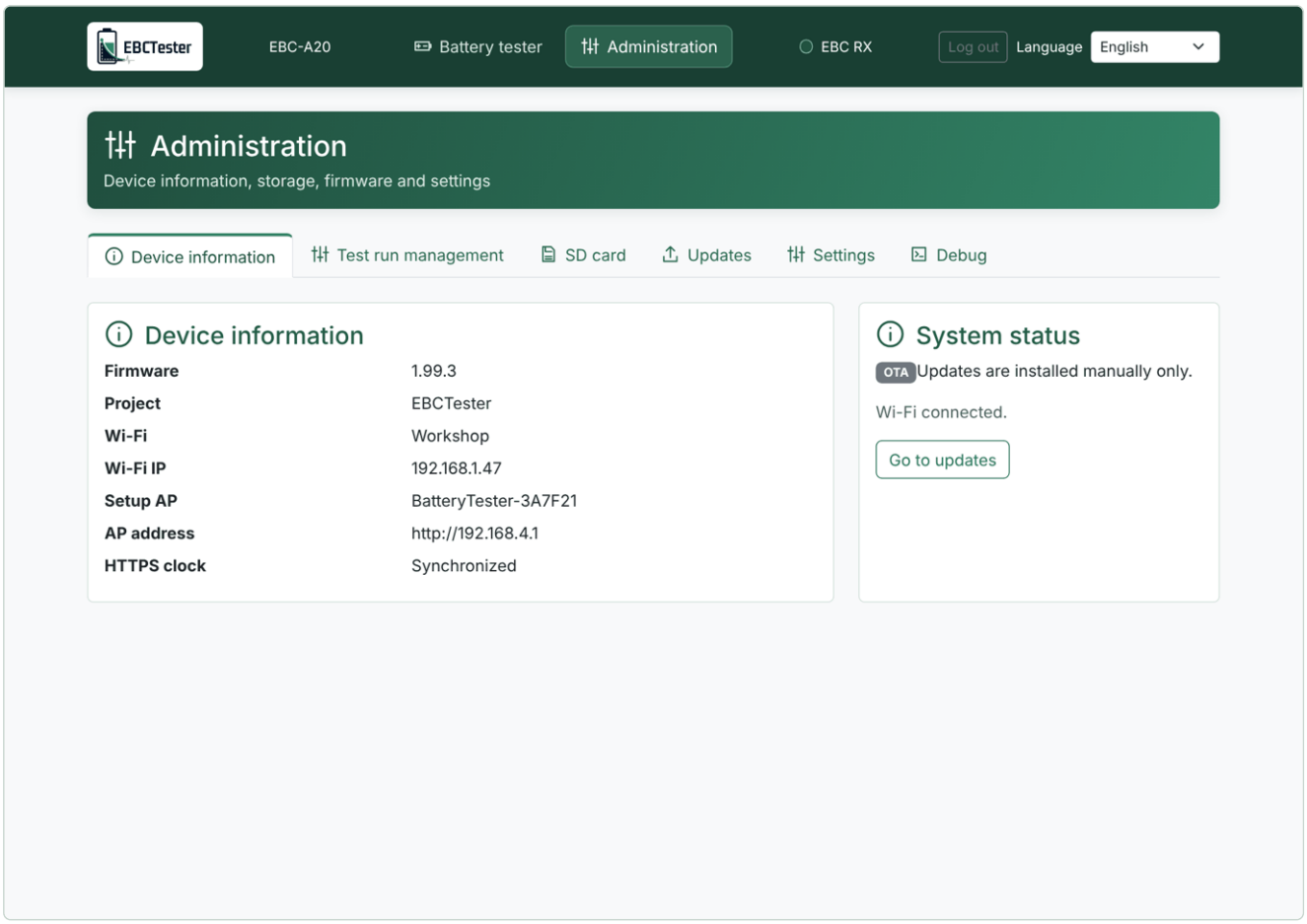



**Figure 11:** Device information

Shows the installed firmware version, the project name, the connected Wi-Fi network and IP address, the name of the setup access point ( `BatteryTester-XXXXXX` ), its address `http://192.168.4.1` , and whether the clock has been synchronised (*HTTPS clock*). **System status** shows the network message and leads to the update page.

## 4.2 Test run management

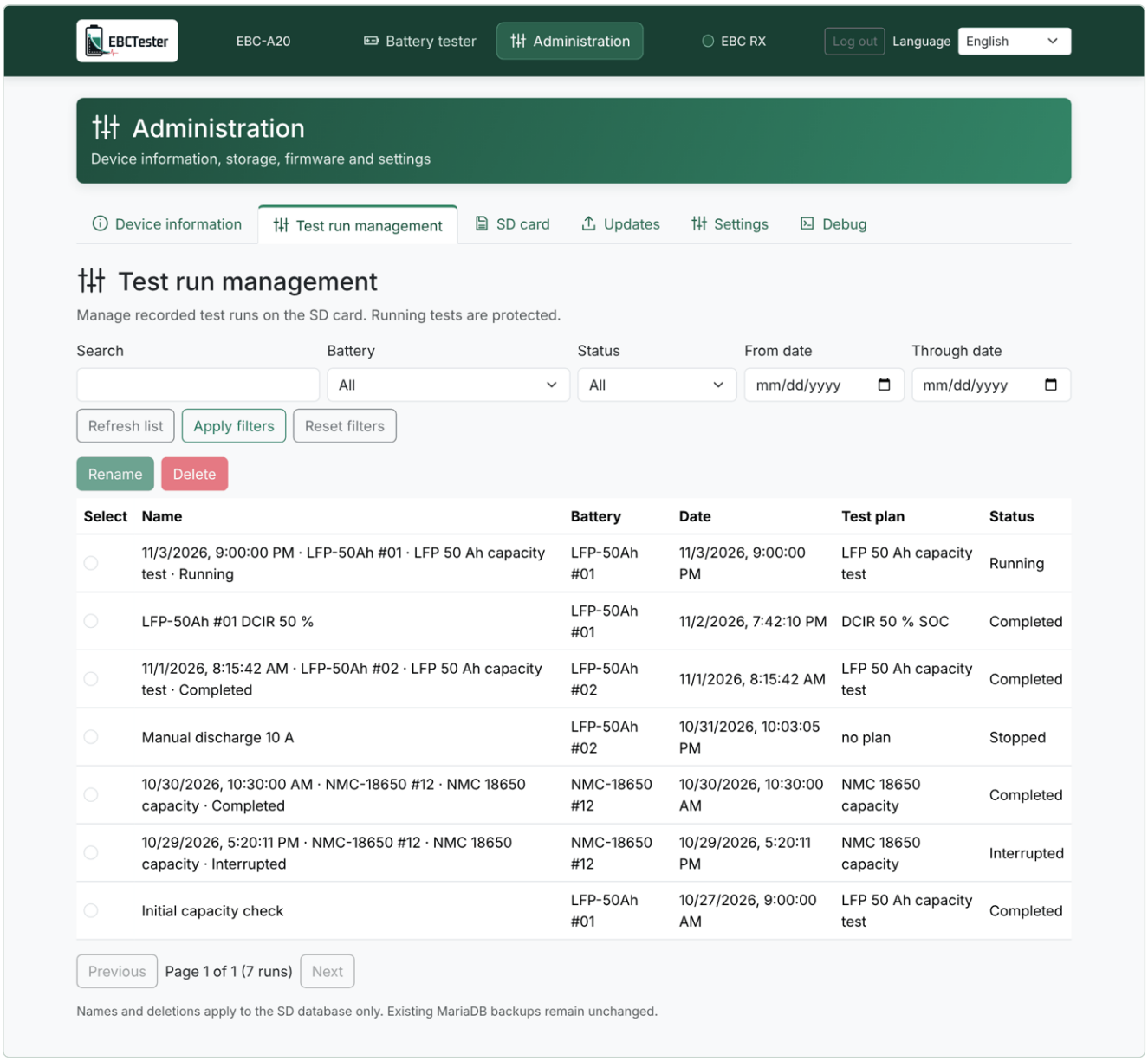



**Figure 12:** Test run management

This table lists all runs in the SD database, 50 per page, newest first.

- **Filter:** free-text **Search** (name, battery, plan, date, status or run ID), **Battery**, **Status** and a date range (**From date** / **Through date**; the end date includes the whole day). Click **Apply filters**; **Reset filters** clears them.
- **Rename:** select one run with the radio button and click **Rename**. Up to 100 characters; an empty name restores the automatic name. The name is also used in the dashboard.
- **Delete:** select a run and click **Delete**. After confirmation the run is permanently removed together with its measurements, segments and DCIR results. Batteries and plans are kept.

Running runs are protected and cannot be renamed or deleted. Renaming and deleting require the login password and only affect the SD database – existing MariaDB backups are not changed, and custom names are not transferred to MariaDB.

> **Caution**
> Deleting cannot be undone. Make a backup (MariaDB synchronisation or a copy of the database file) before deleting important runs, and do not delete runs while a MariaDB synchronisation is running.

## 4.3 SD card

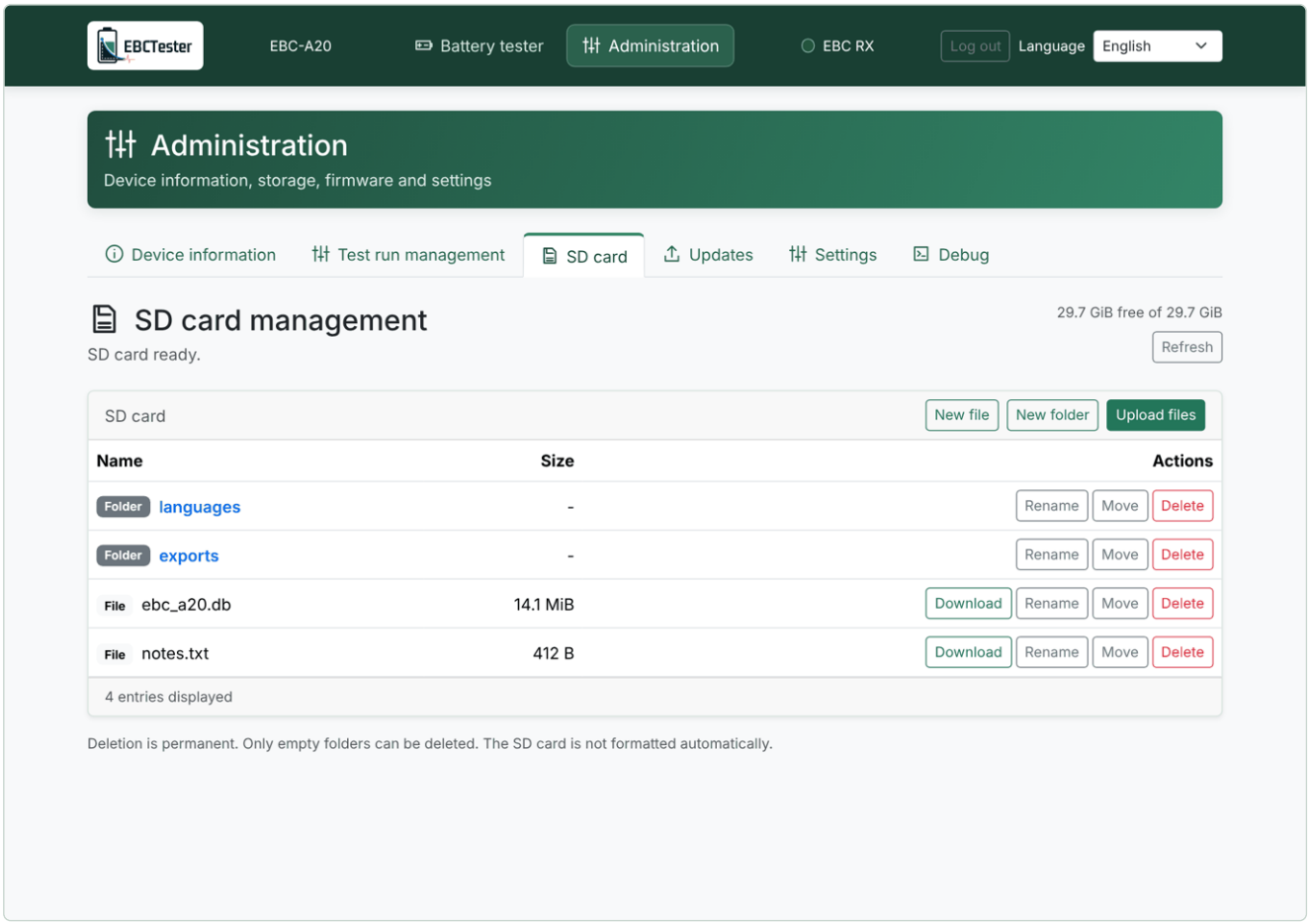



**Figure 13:** SD card management

A simple file manager for the microSD card – no need to remove it from the device. The header shows whether a card is mounted and how much space is free.

- Click a folder name to open it; use the breadcrumb path to go back.
- **New file**, **New folder** and **Upload files** (several files possible, with progress bar).
- For each entry: **Download** (files only), **Rename**, **Move** and **Delete**.

Deleting is permanent; only empty folders can be deleted. The card is never formatted automatically. The measurement database is the file `ebc_a20.db` in the root folder; download it regularly as a backup.

> **Caution**
> Do not rename, move or delete the database file while the device is in use. Download it only when no test is being recorded to get a consistent copy.

## 4.4 Updates

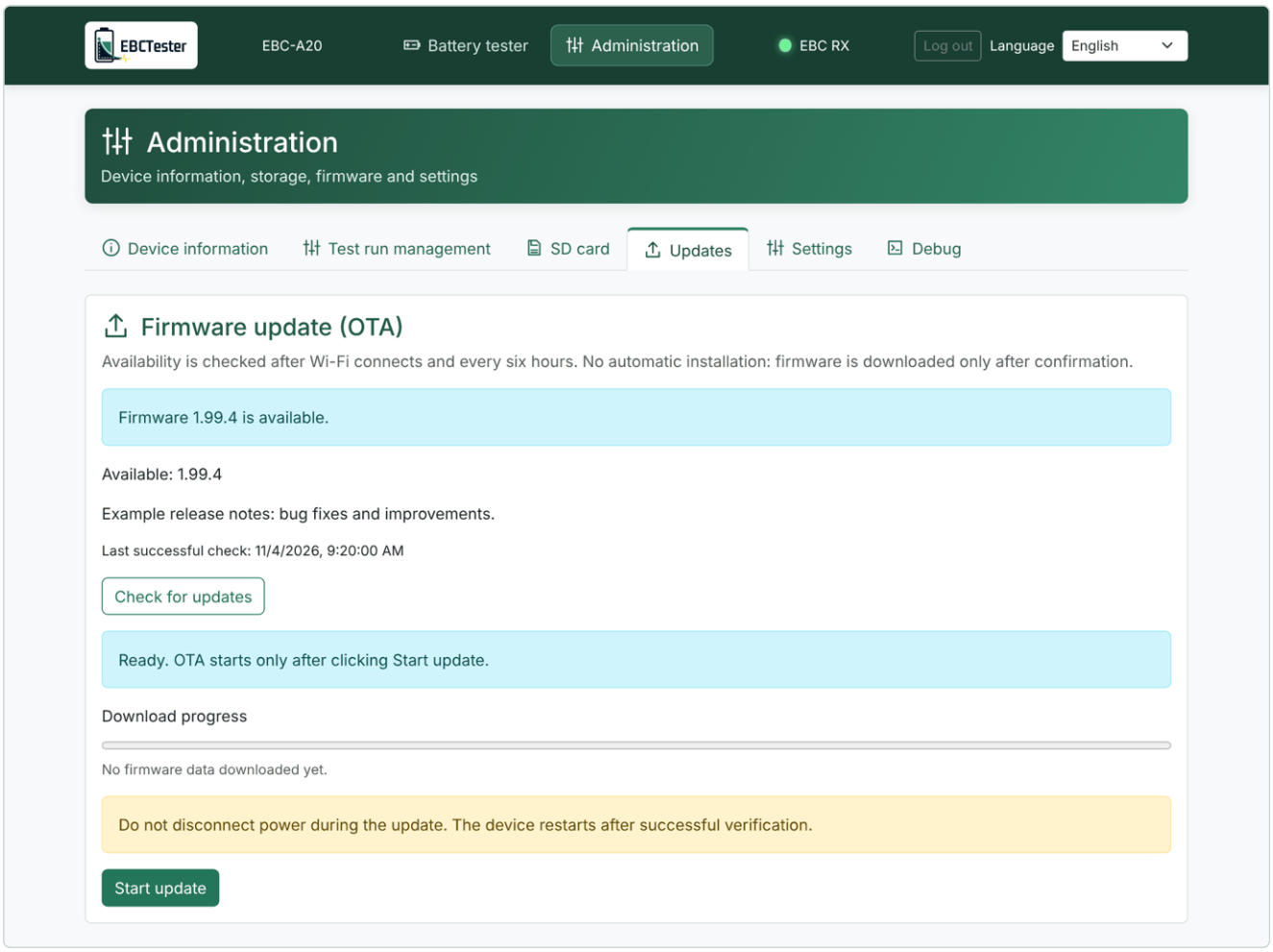



**Figure 14:** Firmware update

When the device is connected to the Internet it checks for new firmware after connecting, every six hours and once each time you open the web interface. Only a small description file is downloaded – firmware is never installed automatically.

1. If an update is available, the new version and its release notes are shown. **Check for updates** repeats the check manually.
2. Make sure no test is running, then click **Start update** and confirm the version.
3. The firmware is downloaded into the inactive memory area and checked (size and SHA-256 checksum). Progress is shown.
4. After successful verification the device restarts. The new firmware is only accepted permanently if it runs stable for 10 seconds; otherwise the previous version is restored automatically.

> **Warning**
> Do not disconnect the power during an update, and never update during a battery test.

A firmware update keeps all settings, including the FactoryMode configuration, the passwords and the Wi-Fi credentials.

## 4.5 Settings

The **Settings** tab collects all device settings. When you save a setting you are asked for the current login password (labelled **Admin password** in the dialog) as an extra protection – also when browser login is switched off. Cancelling or entering a wrong password does not save anything. The access-point password is not used here.

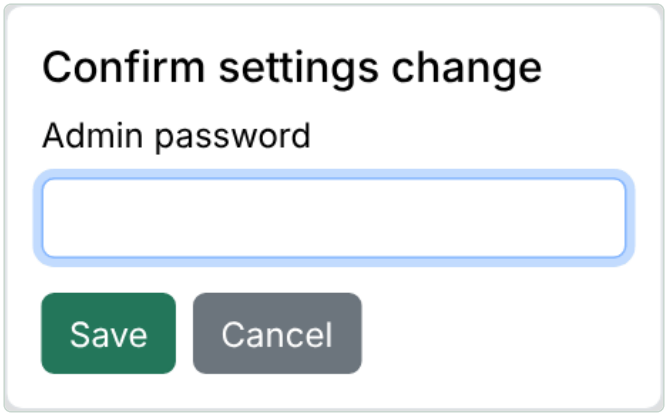



**Figure 15:** Password confirmation when saving settings

### Login Settings

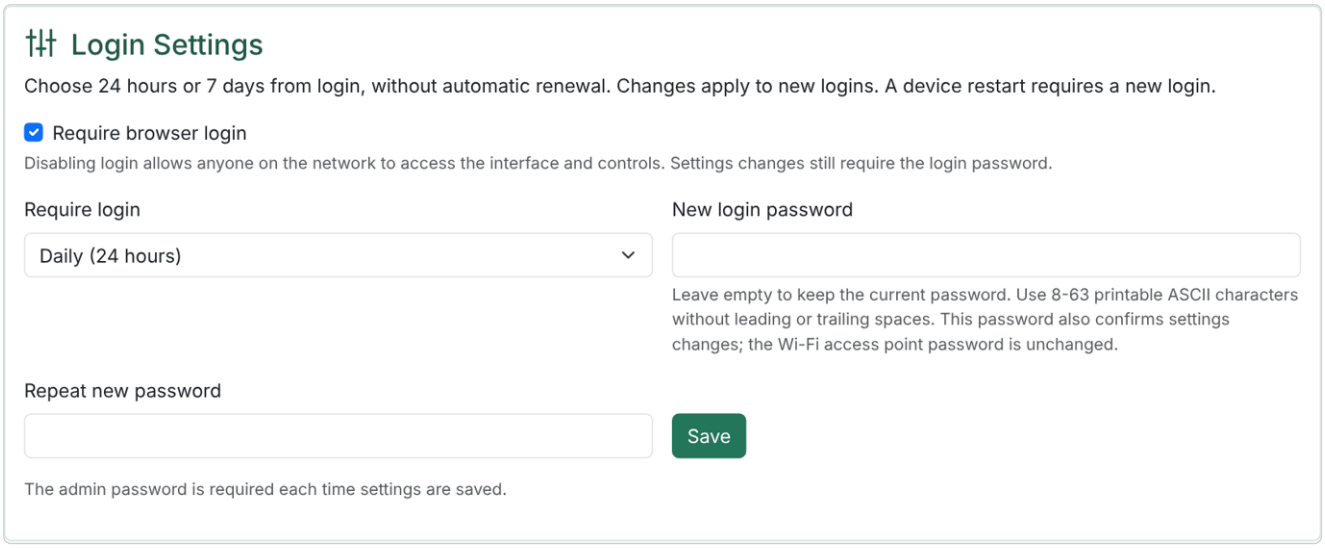



**Figure 16:** Login Settings

| Setting | Description |
|---|---|
| **Require browser login** | On by default. When switched off, the web interface opens without logging in and the **Log out** button disappears. |
| **Require login** | **Daily (24 hours)** (default) or **Weekly (7 days)**. The duration counts from the login, is not extended by activity and applies to new logins. Only available while browser login is on. |
| **New login password / Repeat new password** | Enter the new password twice. 8–63 printable ASCII characters (letters, digits and common symbols, no umlauts), no spaces at the beginning or end. Leave both fields empty to keep the current password. |

Click **Save** and confirm with the current login password – also when you are changing the password itself. The new password is stored permanently on the device and survives restarts and firmware updates. The access-point password is not changed. Changing the password or switching browser login on or off ends all existing sessions; the page reloads automatically. Afterwards log in with the new password.

> **Warning**
> With browser login switched off, everyone in the network can view all data and operate the tester – including starting and stopping tests. Use this only in a network you trust completely. Settings changes still require the login password.

### Configure Wi-Fi

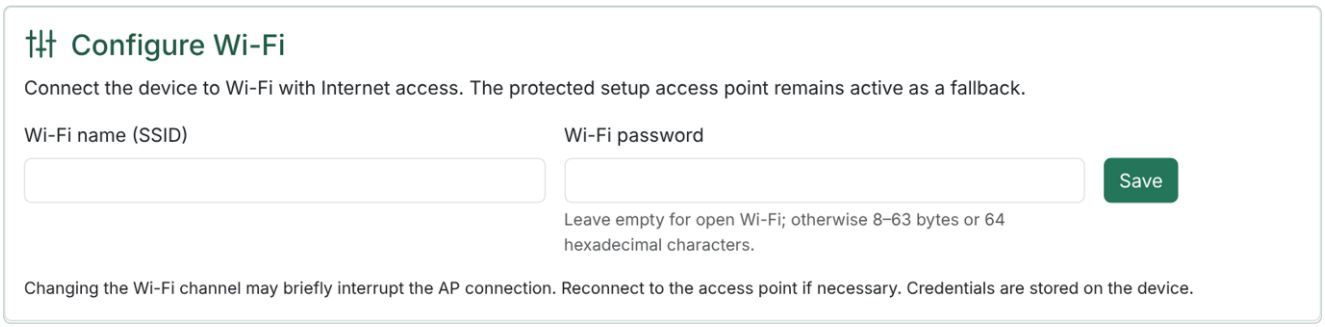



**Figure 17:** Wi-Fi settings

Network name (SSID) and password of your home network. Leave the password empty for an open network; otherwise 8–63 characters or 64 hexadecimal characters. The credentials are stored on the device.

### EBC tester

Selection of the tester model – see Selecting the tester model (section 2.4).

### Home Assistant MQTT

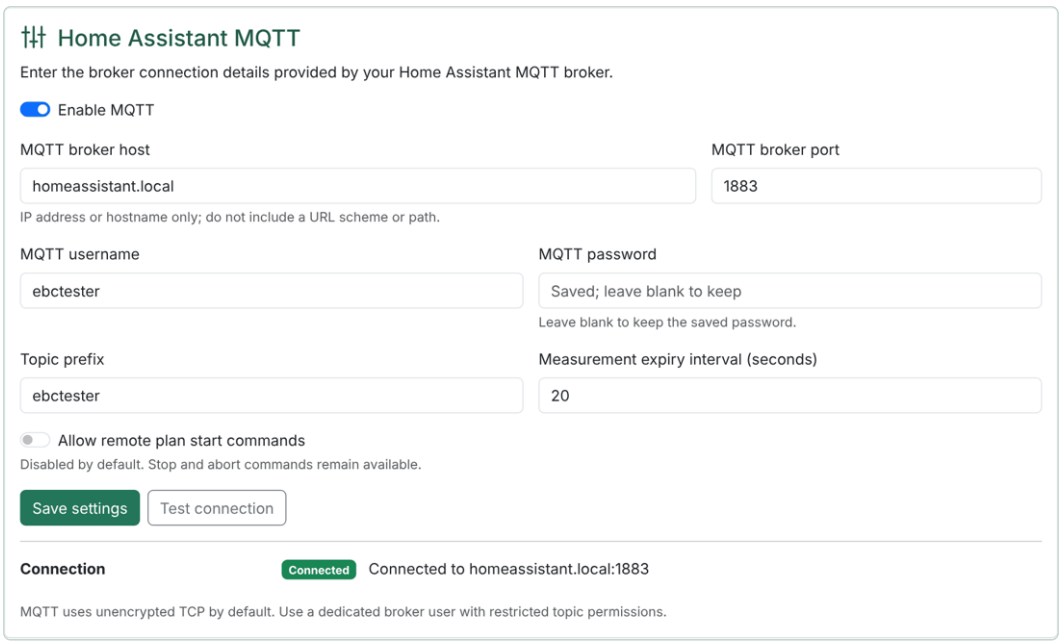



**Figure 18:** Home Assistant MQTT

Optional connection to an MQTT broker, typically the one used by Home Assistant. The device then appears automatically in Home Assistant (MQTT discovery) and publishes voltage, current and capacity of every recorded measurement.

| | |
|---|---|
| **Enable MQTT** | Switches the client on or off. |
| **Broker host / port** | IP address or host name (without `mqtt://`), port usually 1883. |
| **User name / password** | Broker account. Leave the password empty to keep the saved one. |
| **Topic prefix** | Start of all topics, for example `ebctester`. |
| **Measurement expiry interval (s)** | Interval between recorded measurements; values become "unavailable" after three intervals without new data. |
| **Allow remote plan start commands** | Off by default. Only when enabled can a plan be started via MQTT. Stop and Abort always work. |

**Save settings** applies the settings immediately, **Test connection** reconnects with the saved settings. The connection state is shown below. MQTT is unencrypted – use a dedicated broker user with restricted rights. Details are given in Home Assistant MQTT topics (section 12.2).

### MariaDB backup

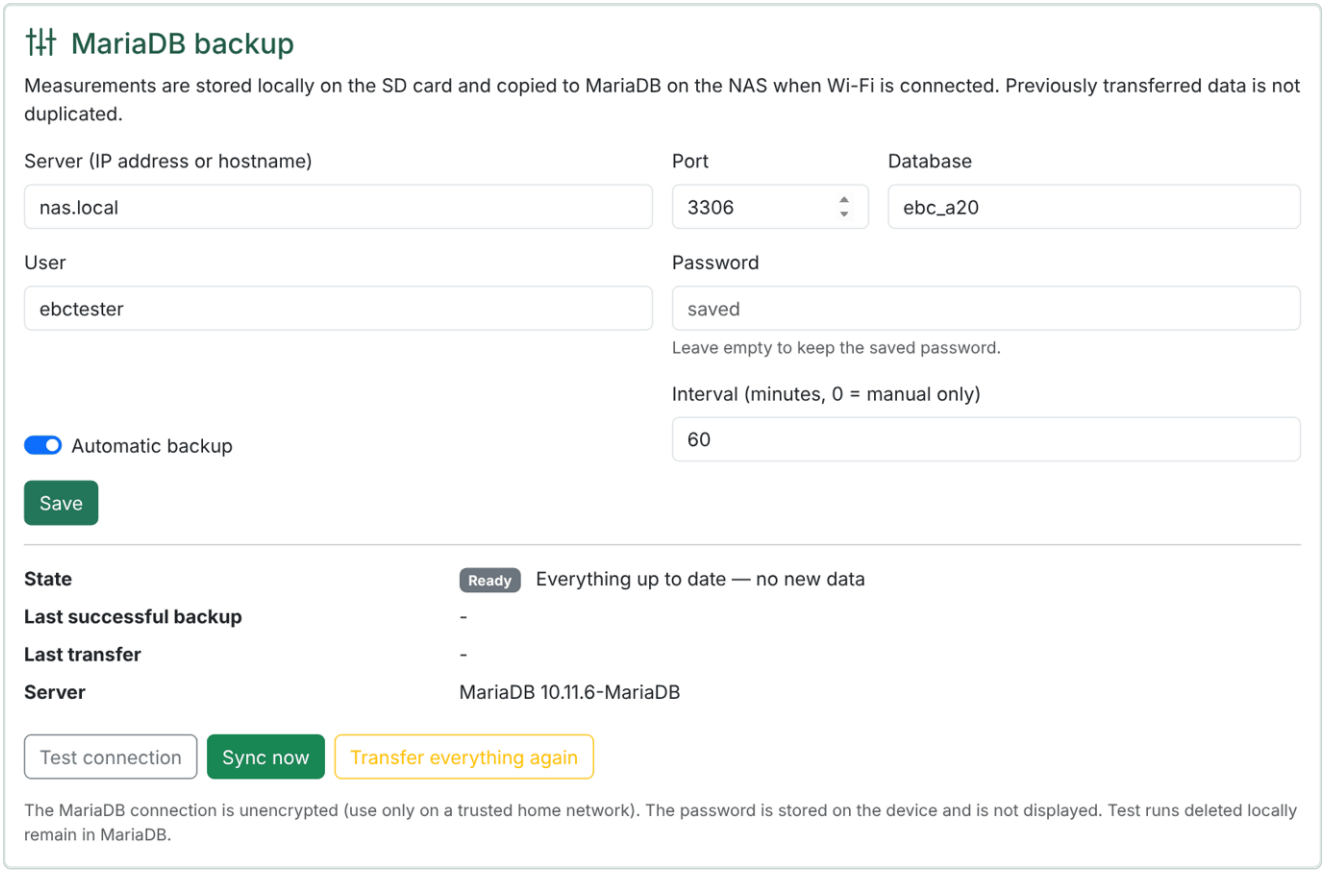



**Figure 19:** MariaDB backup

Copies the recorded test runs to a MariaDB database, for example on a NAS. Only new data is transferred; repeated or interrupted synchronisations do not create duplicates.

- Enter server, port (3306), database, user and password; choose **Automatic backup** and an interval in minutes (0 = manual only). Save.

- **Test connection** checks access, **Sync now** starts a transfer immediately, **Transfer everything again** re-sends all data (no duplicates are created).
- The status area shows state, last successful backup, number of runs and measurements transferred, and the server version.

Automatic backup only runs with Wi-Fi, a ready SD card and a saved password. Runs deleted on the device remain in MariaDB. The database tables must already exist on the server – see MariaDB backup database (section 11.2). The connection is unencrypted: use it only in your home network.

### Debug log

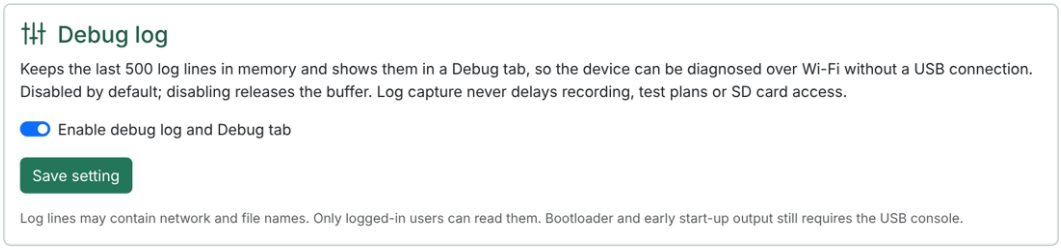



**Figure 20:** Debug log setting

When enabled, the device keeps the last 500 log lines in memory and shows them in the **Debug** tab. Disabled by default; disabling frees the memory. Logging never delays recording, plans or SD access.

### Language




**Figure 21:** Language

English and German are built in. Additional languages can be added as JSON files in the folder `/languages` on the SD card (see Appendix A). **Reload languages** reads the folder again; then select the language in the header.

### Device reset

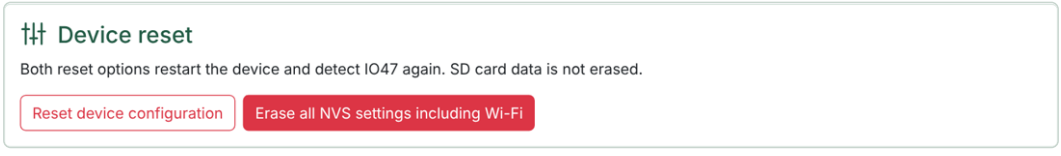



**Figure 22:** Device reset

Two reset functions return the device to the state of its first start. Both ask for confirmation and the login password, restart the device and detect the mode (GPIO 47) again. Data on the SD card – measurements, batteries, plans and language files – is never erased.

| Button | Resets | Keeps |
|---|---|---|
| **Reset device configuration** | Device configuration, login password (back to `ebctester`), login options, access-point password, Firmware mode | Wi-Fi credentials, tester model, current battery and plan, MQTT and MariaDB settings, debug-log setting |

| Button | Resets | Keeps |
|---|---|---|
| **Erase all NVS settings including Wi-Fi** | Everything stored on the device, including Wi-Fi credentials, tester model and all service settings | Only the firmware and the SD card |

A reset is refused while a test, plan or recording is running or a firmware update is in progress. Stop the test and wait until the recording has finished.

> **Note**
> On an original board in its delivered state, the device starts in FactoryMode again after a reset and is immediately ready for use. If JP1 has been changed, the device starts in MakerMode and has to be configured (see the Maker Edition of this manual).

## 4.6 Debug

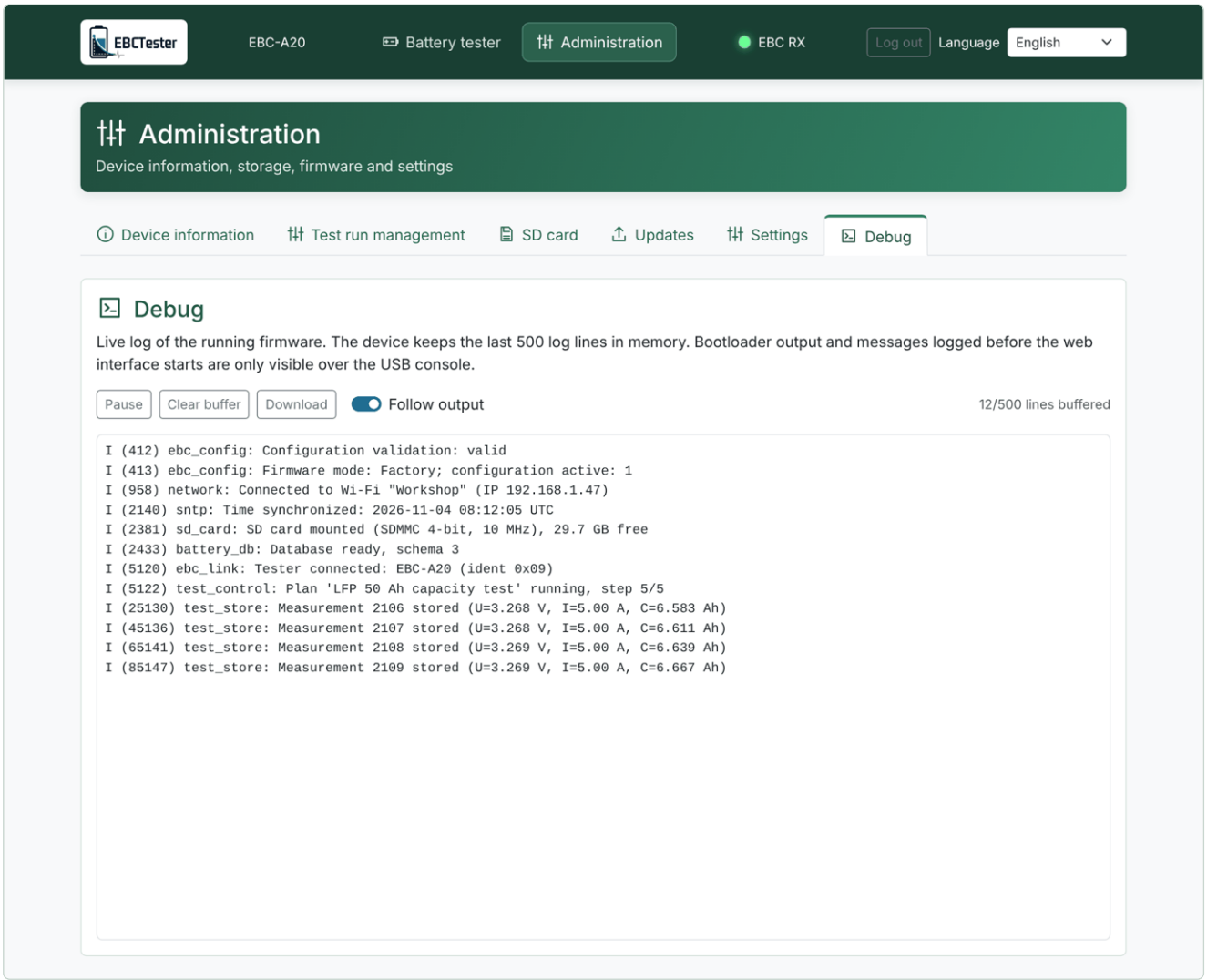



**Figure 23:** Debug tab

Live log of the running firmware – useful when something does not work and no USB cable is at hand. **Pause** freezes the display, **Clear buffer** empties it, **Download** saves the lines as a text file, and **Follow output** keeps the view at the newest line. Log lines may contain network and file names, but never passwords. Messages from the very beginning of the start-up – including the mode detection – are only visible on the USB console.

# 5 Typical workflows

## 5.1 Measuring the capacity of a battery

1. **Battery:** enter the battery data and click **Apply and save to database** (or load a saved battery and click **Apply**).
2. **Plan:** in **Automatic**, load or create a plan: charge (C-CV) – wait – discharge (D-CC) – wait – optionally charge to storage level.
3. **Connect:** in **Manual**, click **Connect** (or enable automatic connection). The RX indicator in the header turns green.
4. **Start:** in **Automatic**, click **Start plan**. You can now close the browser.
5. **Follow:** open **Measurements** at any time. The capacity of the discharge segment in the segment timeline is the measured capacity.
6. **Finish:** when the plan is complete, the run status changes to *Completed*. Give the run a meaningful name in **Test run management** if you like.

## 5.2 Measuring internal resistance at 50 % state of charge

1. Apply the battery as above and connect it with four-wire sensing.
2. In **Internal resistance**, select **Automatic preparation and measurement**, enter the usable capacity (ideally the result of a capacity test), the charge parameters and target SOC = 50 %.
3. Set the pulse current (for example 0.2 C to 1 C, within the battery's limits), check the protection limits, save, tick the confirmation and start.
4. After completion compare the three results. Repeat measurements under the same conditions (temperature, SOC, rest time) to compare batteries.

## 5.3 Backing up your data

- **Simple:** download `ebc_a20.db` in **SD card** when no test is running. The file can be opened with any SQLite tool (for example "DB Browser for SQLite").
- **Automatic:** set up the MariaDB backup. Delete runs on the SD card only after verifying that they have been transferred.

# 6 Troubleshooting

| Problem | Possible cause | Remedy |
|---|---|---|
| Access point `BatteryTester-…` not visible | No antenna, device not powered or still starting | Check the antenna and power; wait 10–20 s after power-on. |
| AP password unknown | Randomly generated at first start, or changed | Read a generated password from the start-up log over USB (serial terminal, press RESET). Otherwise use **Erase all settings over USB** below. |
| Login password not accepted | Password was changed; or the default is still active | Try `ebctester` (default until a password is saved, and after a device reset). A forgotten password can only be reset over USB, see below. |
| Page does not load at 192.168.4.1 | Phone switched back to mobile data or another network | Disable mobile data while connected to the AP; check the Wi-Fi connection. |
| Login keeps returning to the login page | Session expired or device restarted; browser blocks cookies | Log in again; allow cookies for the device address. |
| Login refused although the password is correct | Maximum of 8 simultaneous sessions reached | Log out in other browsers or restart the device. |
| Battery tester section missing; only Settings and Device information shown | Configuration inactive – for example because JP1 was changed and the device started in MakerMode | Check that JP1 is in its delivered state (pads 1–2 bridged), then run **Settings → Device reset → Reset device configuration**. |
| RX indicator stays grey | Tester not connected or wiring wrong | Click **Connect**; check TX/RX crossing and GND; check that the tester is switched on. |
| "Tester mismatch" error | Selected model differs from the connected one | Select the right model in **Settings → EBC tester**, then **Connect**. |
| Start buttons disabled | See the yellow hint below the button | Typical: not connected, waiting for first message, plan running, command pending. |
| "Database not ready" | No card, card not FAT32, card full or defective | Insert a FAT32 card; check the **SD card** tab. exFAT cards must be reformatted to FAT32 on a PC. |
| Times shown as uptime instead of date | Clock not synchronised (no Internet) | Connect the device to Wi-Fi with Internet access. |
| Plan shows "Interrupted" | Power failure or restart during the plan | Check the battery, click **Reset status**, restart the plan if needed. |
| DCIR result "invalid" | Sample too late, voltage drop too small, no current | Increase the pulse current, check four-wire sensing and the timing tolerance. |
| Update check fails | No Internet, clock not yet synchronised, server unreachable | Wait a few minutes after connecting; retry with **Check for updates**. |
| Settings are not saved | Login password wrong or dialog cancelled | Enter the current login password (not the access-point password) in the confirmation dialog. |
| Reset or configuration change refused | A test, plan, recording or update is running | Stop the test and wait until the recording has finished. |

| Problem | Possible cause | Remedy |
|---|---|---|
| Language file not offered | Wrong folder, invalid JSON or wrong code | Place it in `/languages`, check the warning shown in the header, click **Reload languages**. |

### Erase all settings over USB

If neither the login password nor the access-point password is known, the settings memory can be erased with a PC. Firmware and SD card data are kept; all settings return to the state of the first start:

```
esptool.py --chip esp32s3 --port COM5 erase_region 0x9000 0x6000
```

Replace `COM5` with the serial port of the device. If the port is not found, start the device in download mode (hold BOOT, press and release RESET, release BOOT) and press RESET after erasing.

If the problem persists, enable the debug log, reproduce the problem and download the log. When reporting a problem on GitHub, include the firmware version, steps to reproduce and the log – but never passwords or private data.

# Part II – Technical Reference

This part summarises the technical data of the EBCTester board and the firmware: start-up behaviour, connectors, firmware defaults, data storage, interfaces and security. A complete reference for developers and makers is contained in the Maker Edition of this manual.

# 7 System architecture

The firmware is written in C/C++ with Espressif's ESP-IDF 6.0.1 for the ESP32-S3. Tester communication, test control, recording, the charts and the optional services are separate components; networking, firmware updates and SD-card handling are reusable libraries (OTA Suite, SD Card Manager).

The tester sends a status message roughly once per second. `ebc_link` checks each message (checksum, model identifier) and passes it to the controller. While a test is active, `test_store` averages a configurable number of messages (default 20, about one stored measurement every 20 seconds) and writes the measurement in a database transaction to the SD card. Plans and DCIR tests are state machines running in their own tasks; they react to the received messages and send commands to the tester. The web interface only reads data and sends commands through authenticated HTTP requests – it is never needed for a test to continue.

# 8 Start-up and device configuration

## 8.1 Start-up sequence

Before any other service starts, the firmware loads the device configuration from the non-volatile storage (NVS) of the ESP32. Only on a new device, or if the stored configuration is not valid, does it create a new one and detect the firmware mode:

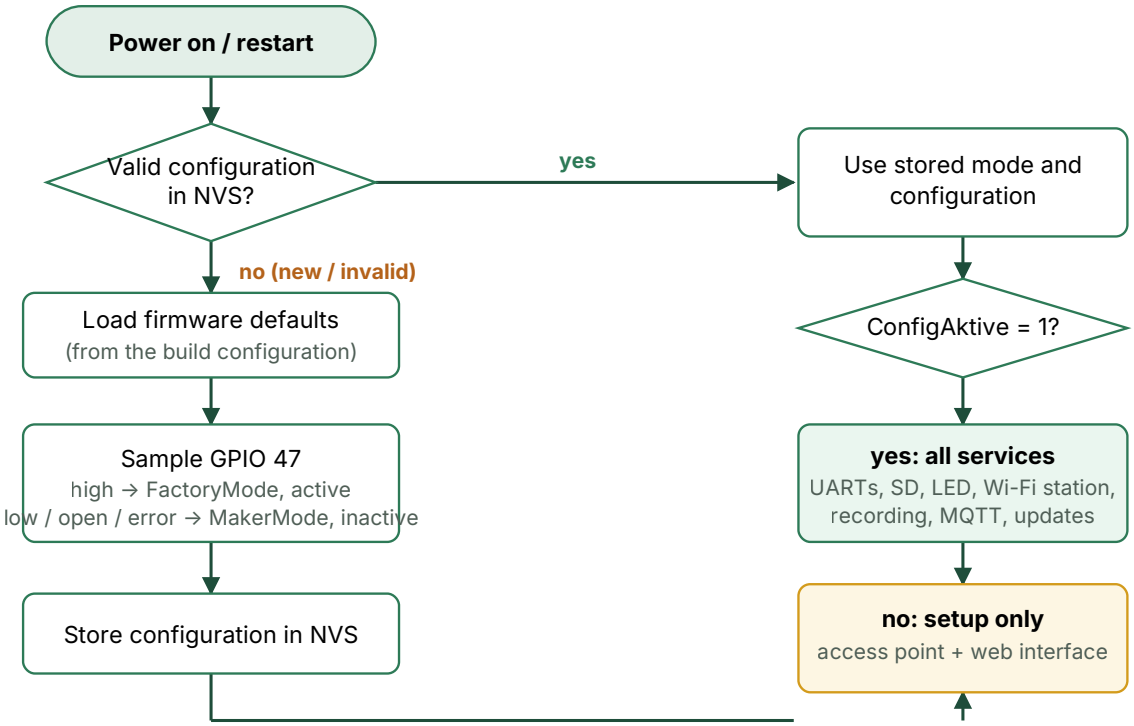



**Figure 24:** Start-up sequence and mode selection

A valid stored configuration always takes precedence: GPIO 47 is evaluated only when a new configuration is created – on a new device, after a device reset or when the stored data is damaged.

## 8.2 Firmware modes

| | FactoryMode | MakerMode |
|---|---|---|
| **Selected by** | GPIO 47 high at the first start | GPIO 47 low, open or inconsistent at the first start |
| **Configuration at first start** | Active (ConfigAktive = 1) | Inactive (ConfigAktive = 0) |
| **Hardware configuration** | Firmware defaults (pin assignment of the original board); not editable in the browser | Firmware defaults as a proposal; edited in **Maker Settings** |
| **Maker Settings tab** | not present | present |
| **Settings-change confirmation** | always on | can be switched off in Maker Settings |
| **Access-point password** | firmware default | editable in Maker Settings |

While the configuration is inactive, no tester or dongle UART, SD card, status LED, Wi-Fi station connection, time synchronisation, update check, recorder or MQTT client is started. Only the access point and the web interface run. The flash memory, the PSRAM and the USB console of the ESP32 are outside this guarantee.

## 8.3 GPIO 47 detection

The level of GPIO 47 is determined without loading the pin permanently:

1. The internal pull-up is switched on; after 200 µs the pin is read.
2. The internal pull-down is switched on; after 200 µs the pin is read again.
3. Steps 1–2 are repeated three times. Afterwards both pull resistors are switched off and the pin is disabled, so no current flows through the internal resistor during operation.

| Read with pull-up | Read with pull-down | Result | Mode |
|---|---|---|---|
| 1 | 0 | open (pin follows the internal resistor) | MakerMode |
| 0 | 0 | low (tied to GND) | MakerMode |
| 1 | 1 | high (tied to 3.3 V) | FactoryMode |
| 0 | 1 | inconsistent (interference) | MakerMode, warning in the log |

The pull-up is "1" only if it is 1 in all three passes; the pull-down result is "1" if it is 1 in any pass. An external connection must be clearly stronger than the internal resistors (about 45 kΩ): use a direct connection or a resistor of at most 10 kΩ. On the original board, solder jumper JP1 ties GPIO 47 directly to 3.3 V (pads 1–2, as delivered) or to GND (pads 2–3).

## 8.4 Configuration storage

All hardware-related settings are stored in the NVS namespace `ebc_cfg`. After the first start the firmware uses only these stored values; the build configuration (sdkconfig) only supplies the defaults for a new configuration.

The configuration is protected by a CRC-32 checksum and a structure version. If the stored data is damaged, the firmware creates a new configuration from its defaults at the next start. Wi-Fi credentials, the tester model, the current battery and plan, and the settings of the optional services are stored separately and are not affected by **Reset device configuration**.

## 8.5 Firmware defaults

A new configuration takes these values from the firmware build. They match the original board.

| Setting | Default | Setting | Default |
|---|---|---|---|
| **Tester UART TX / RX** | GPIO 10 / 11, 9600 Bd | **SD bus** | 4-bit SDMMC, 10 MHz |
| **Dongle UART TX / RX** | GPIO 8 / 9, 9600 Bd | **SD CLK / CMD** | GPIO 6 / 7 |
| **Averaging length** | 20 frames | **SD D0 / D1 / D2 / D3** | GPIO 5 / 4 / 18 / 17 |
| **Device ID** | `EBC-A20` | **Card detect** | GPIO 13 (low = card inserted) |
| **Status LED** | on, GPIO 48 | **Mount point** | `/sdcard` |
| **Access-point password** | `ebctester` | **Login password** | `ebctester`, 24 h, login required |
| **SNTP server** | `pool.ntp.org` | **Settings confirmation** | on |
| **MQTT** | off | **HTTP OTA** | off |

# 9 Hardware

The EBCTester board is a compact controller board. It contains no measurement circuit of its own – all measurement and power electronics are in the ZKE tester. The board provides the microcontroller, power supply, SD card, real-time clock and a level-shifted serial interface.

| | |
|---|---|
| **Microcontroller** | ESP32-S3-WROOM-1U-N16R8: dual-core 240 MHz, 16 MB flash, 8 MB octal PSRAM, Wi-Fi 2.4 GHz with u.FL antenna connector |
| **Power supply** | 19 V via J4 (EBC-A20 adapter) or 5 V via USB-C, both may be connected |
| **Storage** | microSD socket, 4-bit SD mode, card-detect switch |
| **Real-time clock** | DS3231 with CR2032 backup cell (not used by firmware 1.99.3) |
| **Tester interface** | 2 × UART, 5 V TTL via TXS0104E level shifter |
| **Indicators** | Power LED, 2 × WS2812B RGB LED (status, reserved) |
| **Board size** | 80 × 80 mm, 4 × M3 mounting holes (connected to GND) |
| **Revision** | Schematic revision 1 (2 November 2026), PCB marking B1.1, date code 20261103 |

## 9.1 Connectors

### J4 – tester connector (JST XH, 6-pin, 2.5 mm)

| Pin | Signal | Description |
|---|---|---|
| 1 | GND | Ground |
| 2 | VIN_19V | 19 V supply input (from the EBC-A20 power adapter) |
| 3 | UART1_TX_5V | Data from EBCTester to the tester → tester RX |
| 4 | UART1_RX_5V | Data from the tester → tester TX |
| 5 | UART2_TX_5V | Reserved for the PC dongle bridge |
| 6 | UART2_RX_5V | Reserved for the PC dongle bridge |

TX/RX are named from the ESP32's point of view. UART2 is initialised by the firmware but the bridge between the manufacturer's PC software and the tester is not implemented yet.

### J3 – expansion connector (JST XH, 14-pin, 2.5 mm)

| Pin | Signal | Pin | Signal |
|---|---|---|---|
| 1 | VIN_19V | 8 | GPIO 42 |
| 2 | +3.3 V | 9 | GPIO 41 |
| 3 | +5 V | 10 | GPIO 40 |
| 4 | VBUS | 11 | GPIO 39 |
| 5 | RTC_INT (GPIO 14) | 12 | GPIO 38 |
| 6 | I2C_SDA (GPIO 15) | 13 | GND |
| 7 | I2C_SCL (GPIO 16) | 14 | GND |

J3 carries the power rails, the I²C bus with the RTC interrupt and five free GPIOs for extensions (for example a display or temperature sensors). It is not used by the current firmware. Its 3.3 V GPIOs are not 5 V tolerant.

### J5 – LED chain extension (JST XH, 3-pin, 2.5 mm)

| Pin | Signal | Description |
|---|---|---|
| 1 | +5 V | Supply for external WS2812B LEDs (from the +5 V rail, approx. 4.6–4.7 V) |
| 2 | DATA | Data output of LED2 through R16 (470 Ω) |
| 3 | GND | Ground |

External LEDs draw their current from the +5 V rail. With USB-only supply, keep the total current within the 500 mA of the USB port (a WS2812B draws up to approx. 60 mA at full white).

### JP1 – solder jumper "Custom_Settings" (GPIO 47)

| Position | GPIO 47 | Mode at the first start |
|---|---|---|
| Pads 1–2 bridged (as delivered) | high (3.3 V) | FactoryMode |
| Bridge 1–2 cut, pads 2–3 bridged | low (GND) | MakerMode |
| Bridge cut, all pads open | open | MakerMode |

GPIO 47 is not a strapping pin, so every position is safe for booting. A change takes effect only after a device reset (see Device reset (section 4.5)).

## 9.2 GPIO assignment of the board

| Function | GPIO | Function | GPIO |
|---|---|---|---|
| UART1 TX / RX (tester) | 10 / 11 | SD CLK / CMD | 6 / 7 |
| UART2 TX / RX (dongle) | 8 / 9 | SD D0 / D1 / D2 / D3 | 5 / 4 / 18 / 17 |
| I²C SDA / SCL (RTC) | 15 / 16 | SD card detect (low = card) | 13 |
| RTC INT/SQW | 14 | LED1/LED2 data (WS2812B) | 48 |
| JP1 mode selection | 47 | Strapping pins, keep free | 3, 45, 46 |
| USB D− / D+ | 19 / 20 | Expansion J3 | 38 – 42 |
| BOOT | 0 | PSRAM (do not use) | 35 – 37 |
| UART0 RXD0 / TXD0 | 44 / 43 | Not connected | 1, 2, 12, 21 |

# 10 Firmware installation

## 10.1 Installing a release

Each GitHub release contains `EBCTester-<version>-full-install.bin` with bootloader, partition table and application. Flash it at address `0x0` with esptool (see below). Updates are normally installed over Wi-Fi; a USB installation is only needed for the first installation, after a change of the partition table or for recovery.

```
esptool.py --chip esp32s3 --port COM5 write_flash 0x0 EBCTester-1.99.3-full-install.bin
```

Writing the full image does not erase the settings memory. To start like a new device afterwards, use the device reset or the USB erase described in Troubleshooting (chapter 6).

## 10.2 Tester interface

UART1 is connected to the tester with 9600 baud, 8 data bits, odd parity and 1 stop bit (8O1) at 5 V TTL level. Every status frame is checked (frame markers, XOR checksum, model identifier) before its data is used.

# 11 Data storage

## 11.1 SQLite database on the SD card

The database file is `/sdcard/ebc_a20.db`. It is created when a card is mounted; if the device starts without a card, it is created as soon as one is inserted. The card is never formatted and an existing database is never deleted. A file without a valid SQLite header is preserved under the name `ebc_a20.db.invalid-XXXXXXXX` before a new database is created.

| Table | Content |
|---|---|
| ebc_battery | Batteries (label, manufacturer, serial number, date, nominal capacity, notes) |
| ebc_test_plan / ebc_test_plan_step | Saved test plans and their steps |
| ebc_test_run | Test runs (battery, plan, start/end, status, current step, cycle, firmware, notes, display name) |
| ebc_run_segment | Segments of a run (mode, step, cycle, set-points) |
| ebc_measurement | Measurements (time UTC and uptime, current, voltage, capacity …) |
| ebc_resistance_result | Three DCIR results per test (times, U1/I1, U2/I2, mΩ, validity) |

Time stamps are stored in UTC and require a synchronised clock (SNTP). Without it, the device uptime is recorded instead; no calendar time is invented.

## 11.2 MariaDB backup database

The MariaDB synchroniser copies plans, steps, batteries, runs, segments, measurements and DCIR results to an existing MariaDB schema (for example `ebc_a20`). Progress is tracked per run; only new data is sent.

- The base schema (migrations 001–005) must exist on the server. The repository contains migration `sql/006_resistance_results_mariadb.sql` for DCIR results, not a complete initial schema.
- The connection is plain TCP without TLS.

# 12 Interfaces

## 12.1 HTTP API

The web interface uses a JSON API that can also be used by your own scripts. With browser login enabled, all endpoints require a login session (cookie); POST requests additionally require the CSRF token from `GET /api/status` in the header `X-CSRF-Token`. Saving settings also requires `X-Admin-Password` when the password confirmation is enabled.

| Endpoint | Method | Purpose |
|---|---|---|
| /api/auth/login, /api/auth/logout | POST | Login (JSON user name/password, header `X-Web-Login: 1`) and logout |
| /api/status | GET | Device, network and update status, CSRF token, `configActive`, `firmwareMode` (`factory` / `maker`), tabs |
| /api/tester/status | GET | Live values, connection, model and limits, plan, recorder |
| /api/tester/command | POST | connect, disconnect, stop, auto_connect, charge, discharge_cc, discharge_cp |
| /api/tester/plan, /api/tester/plan/action | POST | Save plan; start, abort, reset, load_db, save_db |
| /api/db/runs, /api/db/run?id=, /api/db/measurements?id=&max= | GET | Run catalogue, run details and segments, measurement series |

## 12.2 Home Assistant MQTT topics

On connection the device publishes a retained discovery document to `homeassistant/device/ebctester_<MAC>/config`, its availability (`online` / `offline` as Last Will) to `<prefix>/availability`, and every recorded measurement (voltage, current, capacity) as a non-retained QoS 0 message to `<prefix>/state`. Mode, plan status and battery entities are already announced but do not yet receive values. Commands are accepted on `<prefix>/command` as a JSON object:

```
{"command":"start_plan"}        (only if remote start is allowed)
{"command":"stop"}
{"command":"abort"}
{"command":"select_plan","name":"Plan name"}
{"command":"select_battery","name":"Battery label"}
```

MQTT cannot set free set-points or create database entries. Invalid commands are ignored. Failed connections are retried with increasing intervals from 1 to 60 seconds.

# 13 Security and limitations

## 13.1 Security

- **Login:** user `admin` with the login password (initially `ebctester`). The password, the session duration and the login switch are stored in the device configuration. Sessions use a random 256-bit cookie (`HttpOnly`, `SameSite=Lax`), last exactly 1 or 7 days and are held in RAM (a restart ends them). At most 8 sessions at a time; failed logins are delayed by one second.
- **Settings protection:** all changing requests require a CSRF token; settings additionally require the current login password, independent of whether browser login is enabled. Password or login-policy changes invalidate all sessions.
- **Access point:** the access-point password is defined by the firmware and cannot be changed in the browser on a Factory device. Keep the access point in a trusted environment and connect the device to your own Wi-Fi.
- **Transport:** the web interface, MQTT and MariaDB are unencrypted. Use only trusted networks, never forward port 80 to the Internet.
- **Storage:** NVS and flash are not encrypted by default; protect physical access and the USB port (the start-up log shows a generated access-point password). Change the shared initial login password `ebctester` on first use.
- **Firmware:** SHA-256 protects against corruption, not against deliberate replacement. Secure Boot, signed images and flash encryption are separate deployment decisions.

## 13.2 Known limitations (firmware 1.99.3)

- Firmware 1.99.3 is a beta version of V2. The hardware acceptance tests on the original board are still pending.
- The UART2 bridge for the manufacturer's PC software is not implemented.
- The DS3231 real-time clock is not used yet; date and time come from SNTP only.
- Only EBC-A20, EBC-A40L and EBC-A50L can be controlled; physical tests with the A50L are pending.
- No battery-chemistry profiles and no temperature measurement.
- MQTT publishes measurements only; status entities do not yet receive values.
- LED2 of the original board is not driven yet.
- A forgotten login password can only be reset by erasing the settings memory over USB.

## 13.3 License

Copyright 2026 Thomas Stoll. EBCTester, OTA Suite and SD Card Manager are licensed under the Apache License 2.0 – commercial use, modification and sale are permitted under its terms; its warranty and liability disclaimers apply. Third-party components keep their own licenses: Bootstrap 5.3.3 and Chart.js 4.4.9 (MIT), SQLite (public domain), ESP-IDF (Apache 2.0) and others, listed in `THIRD_PARTY_NOTICES.md`. EBCTester is an independent project and not an official ZKETECH product; product names are trademarks of their respective owners.

# Appendix A – Language files

Additional user-interface languages are loaded from the folder `/languages` on the SD card. A complete template is `sd_card/languages/en-custom.json` in the repository; a partial Filipino (Tagalog) example is `fil.json`.

1. Copy the template, for example as `fr.json`.
2. Set `"code": "fr"` and `"name": "Français"`.
3. Translate only the values in `"strings"`; keep all keys unchanged.
4. Keep placeholders such as `{p0}`, `%u`, `%s` (printf placeholders in the same order).
5. Save as UTF-8 without BOM, upload to `/languages/fr.json` (for example in the SD card tab), click **Reload languages** and select the language in the header.

```
{
  "version": 1,
  "code": "fr",
  "name": "Français",
  "strings": {
    "language.title": "Langue",
    "...": "..."
  }
}
```

Missing keys fall back to English. Files named `en.json` or `de.json` replace the built-in texts while present. Limits: max. 16 files, 128 KB per file, 1024 entries, 4096 bytes per value, 80 bytes for the name. Language codes use lower-case letters, digits and hyphens (max. 24 characters), for example `fr`, `en-gb`.

# Appendix B – Glossary

| | |
|---|---|
| **AP** | Access point – the device's own Wi-Fi network for setup and fallback access. |
| **C-CV** | Charging with constant current, then constant voltage (CC/CV). |
| **ConfigAktive** | Stored switch that enables the hardware configuration. When off, only the access point and the web interface run. |
| **CSRF** | Cross-site request forgery; the token protects against requests from foreign web pages. |
| **D-CC / D-CP** | Discharging with constant current / constant power. |
| **DCIR** | Direct-current internal resistance, measured with a load pulse. |
| **FactoryMode** | Firmware mode of the original board (GPIO 47 high): ready to use with the firmware defaults. |
| **Frame** | One message between EBCTester and the tester. |
| **MakerMode** | Firmware mode of self-built devices (GPIO 47 low or open): hardware configured in Maker Settings. |
| **NVS** | Non-volatile storage in the ESP32 flash for settings. |
| **OTA** | Over-the-air – firmware update over Wi-Fi. |
| **Segment** | Section of a test run with one operating mode. |
| **SNTP** | Simple Network Time Protocol – time synchronisation over the Internet. |
| **SOC** | State of charge in percent. |
| **Test run** | Recording of one manual operation, plan or DCIR test with all its measurements. |